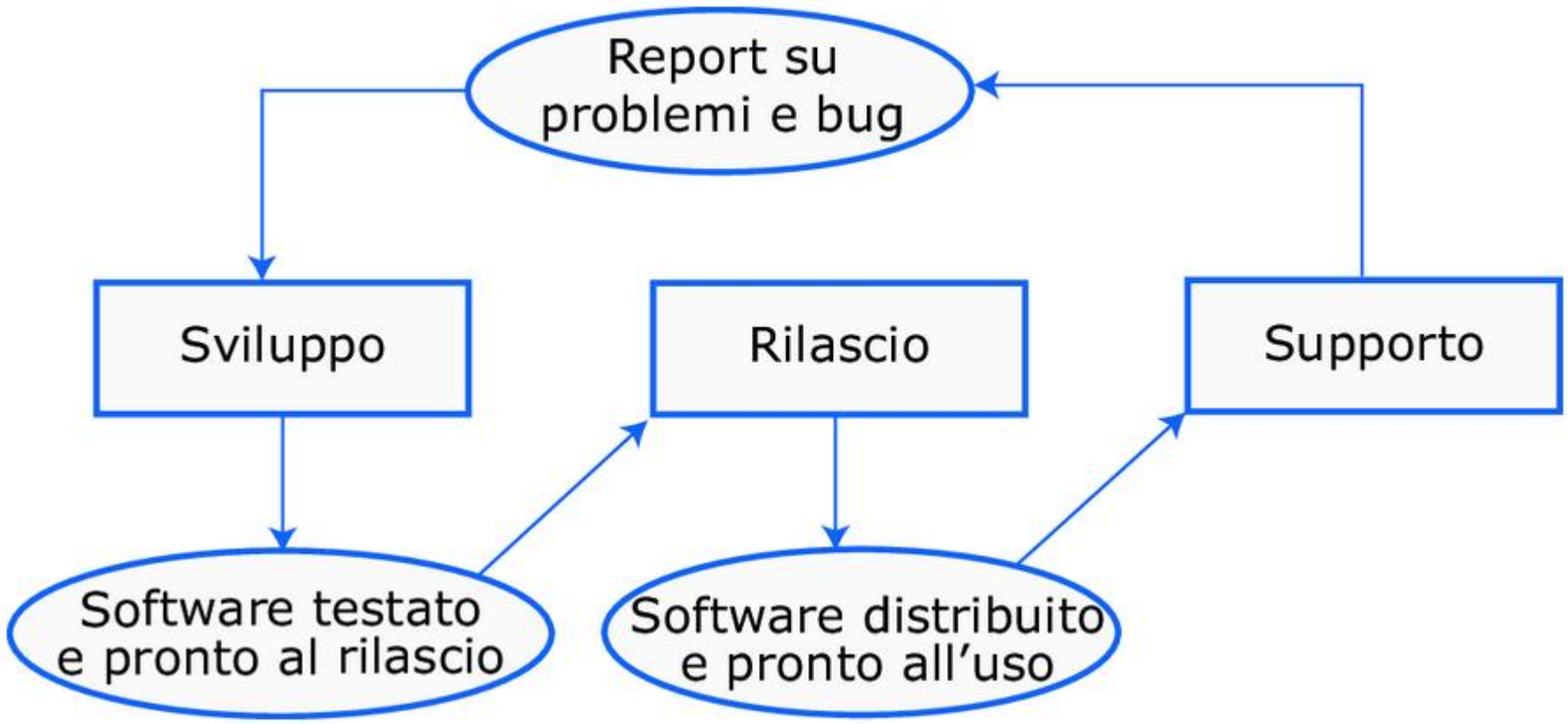
DevOps/SecDevOps e gestione del codice

Elvinia Riccobene

Supporto al rilascio del software



Attività di sviluppo, relascio e supporto affidate a team distinti

Supporto al rilascio del software

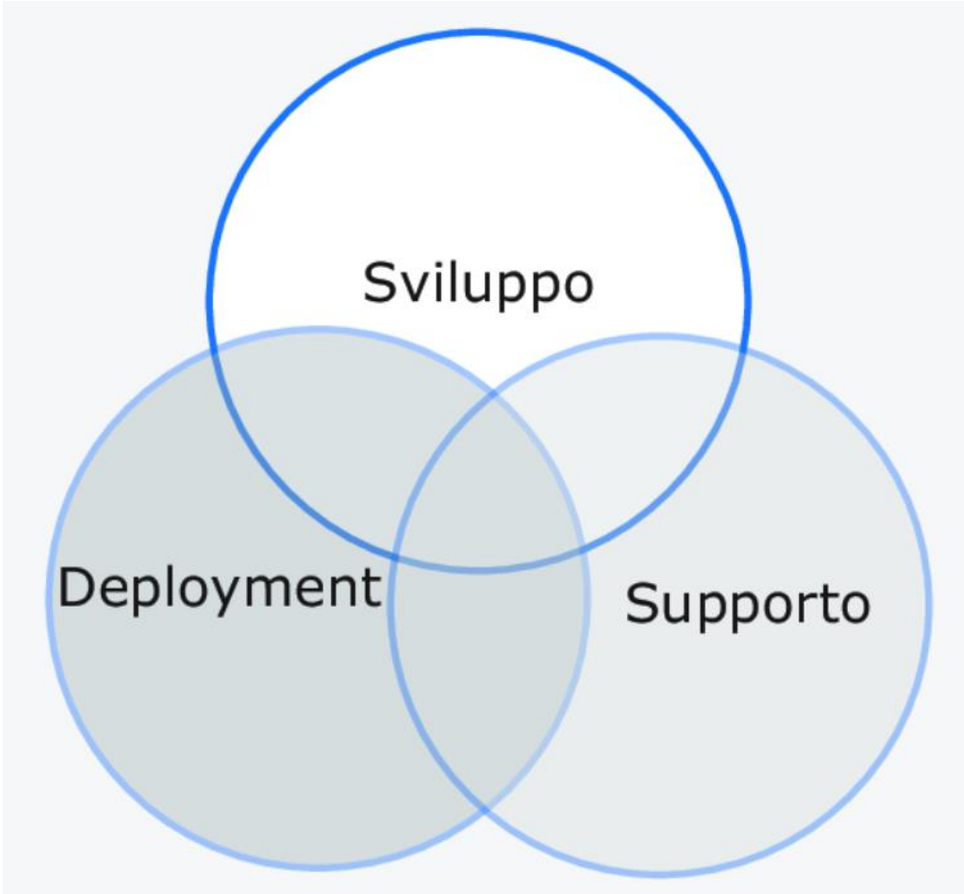
- Approccio tradizionale:
  - Il **team di sviluppo** sviluppa la versione "finale" del software che passa al team di rilascio.
  - Il **team di rilascio** crea una versione di rilascio, la testata e prepara la documentazione di rilascio prima di rilasciare il software ai clienti.
  - Il **team di assistenza** clienti si occupa della successiva fase di supporto.
    - In caso di modifica correttiva/adattativa:
      - modifiche gestite da un "team di manutenzione" separato (raramente quello di sviluppo)

Limiti del processo tradizionale

- Ritardi per problemi di comunicazione
- Scarsa comprensione dei requisiti
- Uso di strumenti diversi tra team diversi
- Lunghi tempi di attesa nella gestione delle versioni corrette o per gestire vulnerabilità

DevOps

- **DevOps** (Development+Operations) sviluppato per velocizzare i processi di sviluppo, rilascio e supporto



DevOps: perché è nato

- **DevOps** (Development+Operations) sviluppato per velocizzare i processi di sviluppo, rilascio e supporto
- Fattori chiave per lo sviluppo e la diffusione di DevOps:
  - I processi agili del software
  - Sviluppo a microservizi: riprogettazione del *software come servizi* con amento dell'affidabilità (Amazon)
  - SW rilasciato *come servizio* in esecuzione su un *cloud pubblico o privato*

Principi DevOps

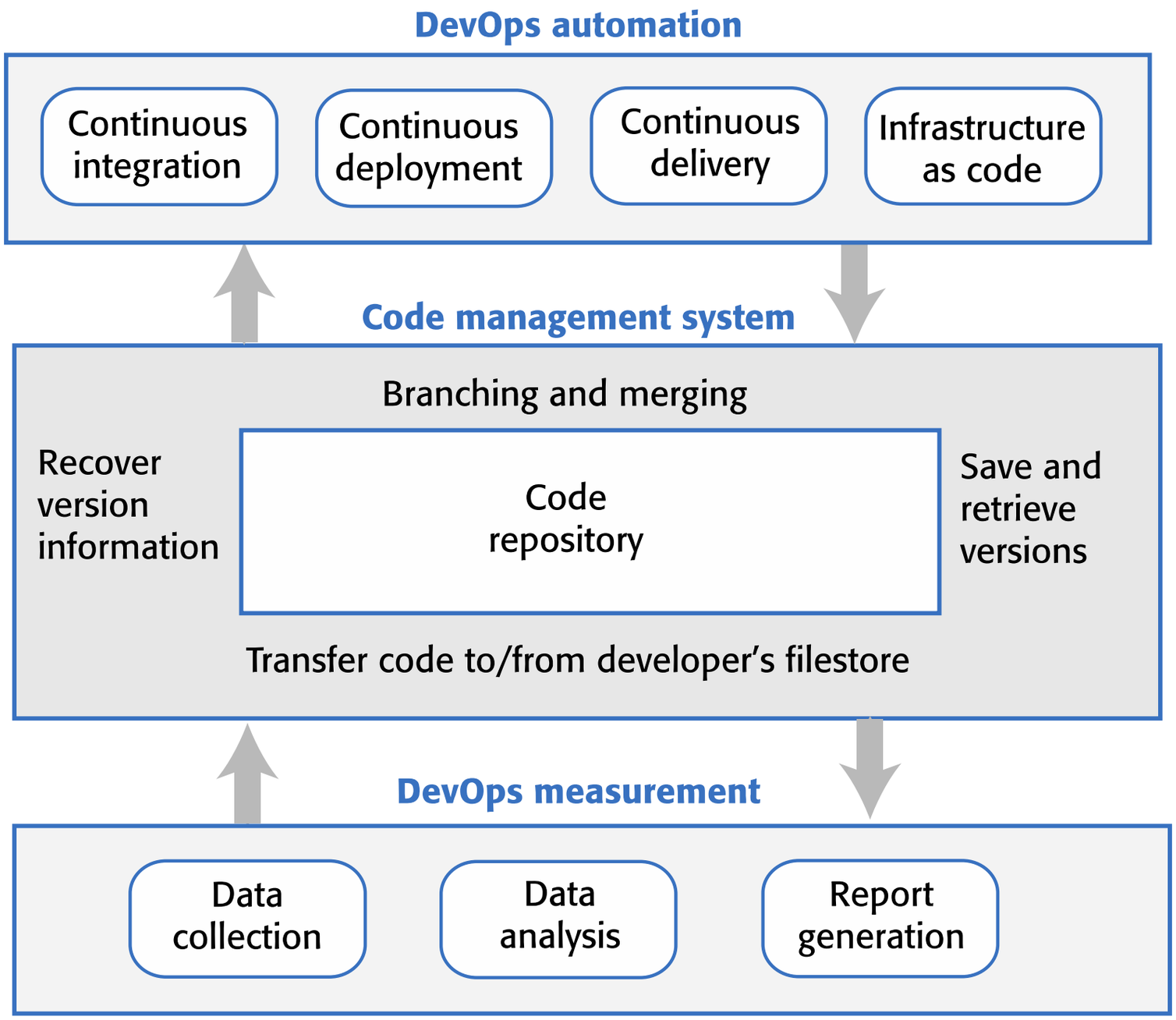
- **Ognuno è responsabile di tutto il processo**
  Tutti i membri del team hanno la responsabilità congiunta per lo sviluppo, la consegna e il supporto del software.
- **Tutto ciò che può essere automatizzato dovrebbe essere automatizzato**
  Tutte le attività coinvolte nei test, nell'implementazione e nel supporto dovrebbero essere automatizzate se possibile→ *minimo coinvolgimento manuale nel deployment*.
- **Prima misura, poi modifica**
  DevOps guidato da programmi di *misurazione* in cui si raccolgono dati sul sistema e sul suo funzionamento.

I dati raccolti sono usati per prendere decisioni su <u>cambiamenti dei processi</u> e degli <u>strumenti</u> DevOps.

Vantaggi di DevOps

- ***Implementazione più rapida***
  Il software può essere implementato in produzione più rapidamente perché i ritardi di comunicazione tra le persone coinvolte nel processo sono drasticamente ridotti.
- ***Rischio ridotto***
  L'incremento di funzionalità in ciascuna versione è ridotto, pertanto vi sono minori possibilità che interazioni tra funzionalità e altre modifiche causino guasti e interruzioni del sistema.
- ***Riparazione più rapida***
  I team DevOps collaborano per ripristinare il funzionamento del software il prima possibile. Non è necessario scoprire quale team è responsabile del problema e attendere che lo risolvano.
- ***Team più produttivi***
  I team DevOps sono più soddisfatti e produttivi rispetto ai team coinvolti nelle attività separate.

Strumenti e operazioni di DevOps



Gestione del codice

- Un prodotto software aziendale consiste di
  - Decine di migliaia di righe di codice e test case
  - File e test sono organizzati in centinaia di file
  - Dozzine di librerie e programmi esterni possono essere coinvolti nella creazione e nell'esecuzione del codice
- Lo sviluppo parallelo del codice può causare conflitti, inconsistenze, mancanza di affidabilità
- Vediamo uno scenario

Un problema di gestione del codice

- Alice e Bob lavoravano per un'azienda chiamata **FinanceMadeSimple** ed erano membri del team coinvolti nello sviluppo di un prodotto finanziario personale.
- Alice ha scoperto un bug in un modulo chiamato **TaxReturnPreparation**. Il bug era che una dichiarazione dei redditi veniva segnalata come archiviata ma, a volte, non veniva effettivamente inviata all'ufficio delle imposte. Ha modificato il modulo per correggere il bug.
- Bob stava lavorando all'interfaccia utente per il sistema e stava anche lavorando a **TaxReturnPreparation**. Sfortunatamente, ne prese una copia prima che Alice risolvesse il bug e, dopo aver apportato le modifiche, salvò il modulo. Questo ha sovrascritto le modifiche di Alice ma lei non ne era a conoscenza.
- I test del prodotto non hanno rivelato il bug in quanto si trattava di un errore intermittente che dipendeva dalle sezioni del modulo di dichiarazione dei redditi che erano state completate.
- Il prodotto è stato rilasciato con il bug.
- Per la maggior parte degli utenti, tutto ha funzionato bene. Tuttavia, per un piccolo numero di utenti, la dichiarazione dei redditi non è stata presentata e sono stati multati dall'Agenzia delle Entrate. L'indagine successiva ha dimostrato che la società di software era negligente. Questo è stato ampiamente pubblicizzato e, oltre a una multa da parte delle autorità fiscali, gli utenti hanno perso fiducia nel software. Molti sono passati a un prodotto concorrente. FinanceMade Simple fallì e sia Bob che Alice persero il lavoro.

Gestione del codice e DevOps

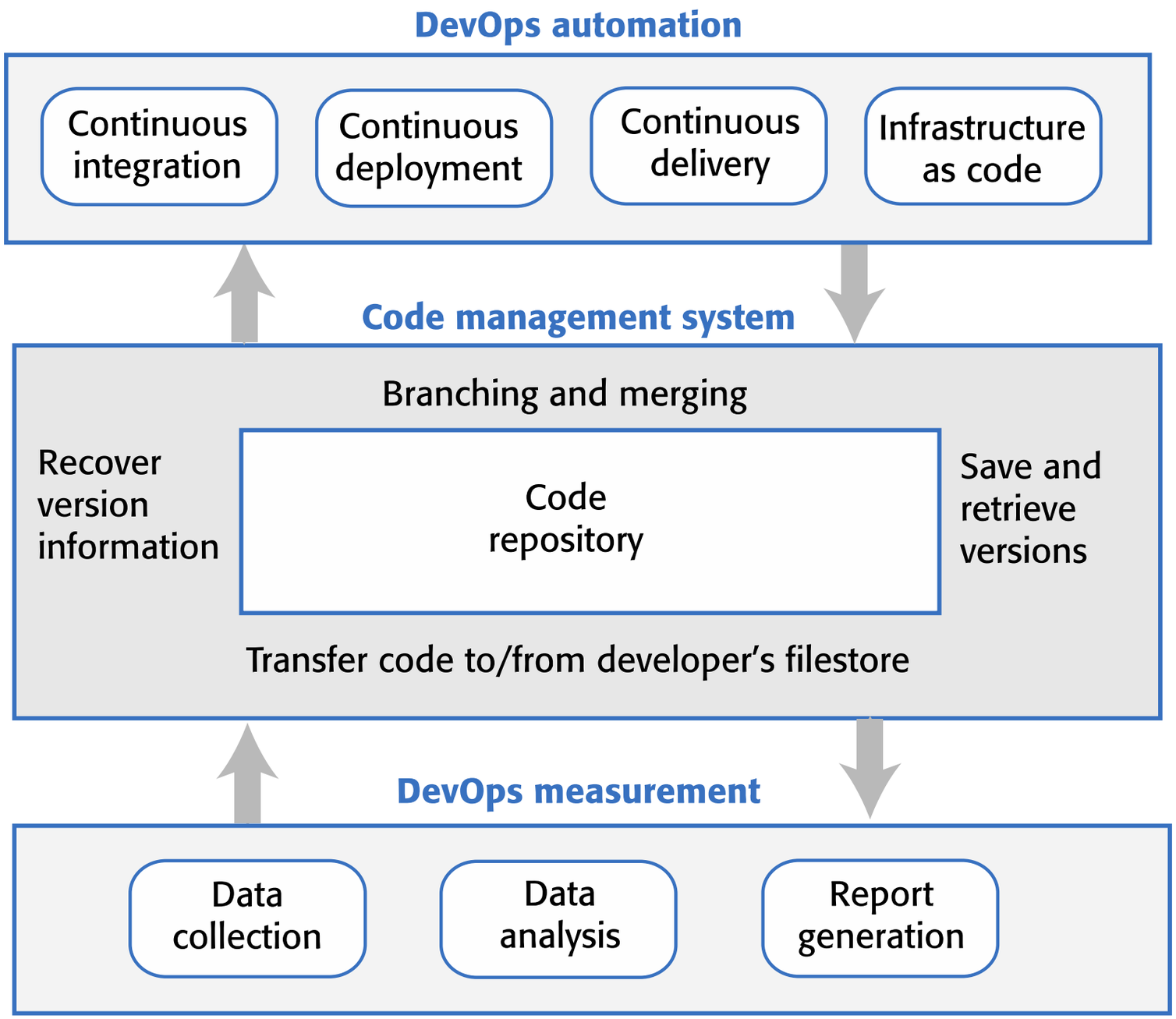
Per **gestione del codice** in DevOps si intende un insieme di pratiche (supportate da software) utilizzate per gestire l'evoluzione del codice.

In linea generale si utilizzano:

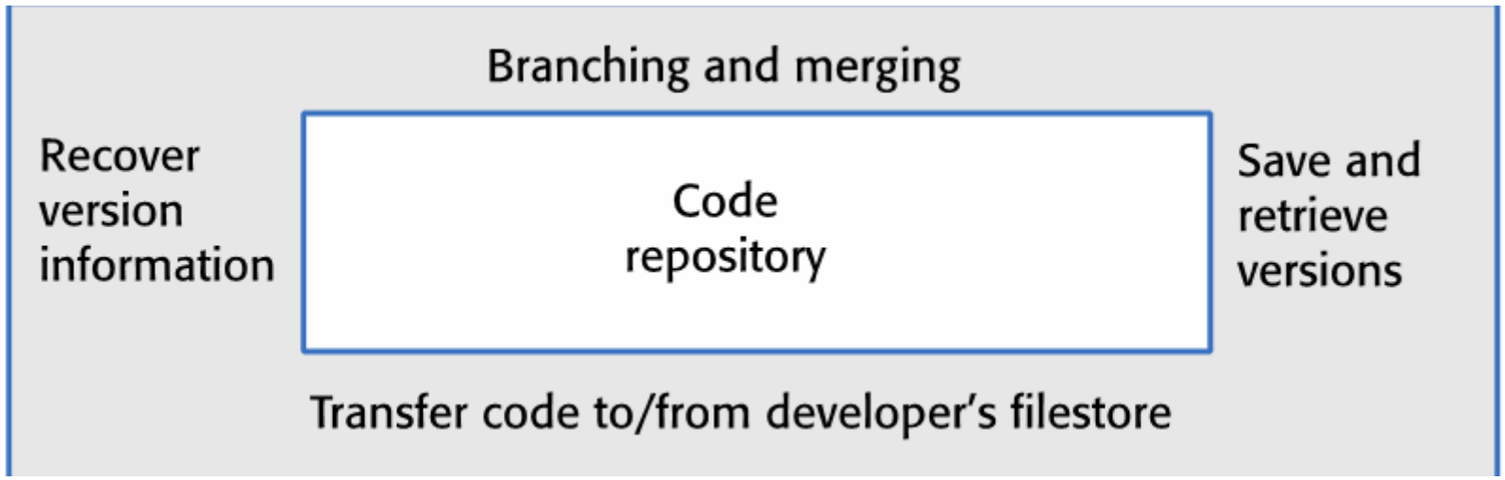
- Strumenti per il code versioning
- Strumenti di continuos integration

<u>Strumenti di automazione e misurazione</u> DevOps interagiscono con il sistema di gestione del codice

Gestione del codice e Devops



Fondamenti di gestione del codice



I sistemi di gestione del codice forniscono una serie di **funzionalità** che supportano quattro aree generali:

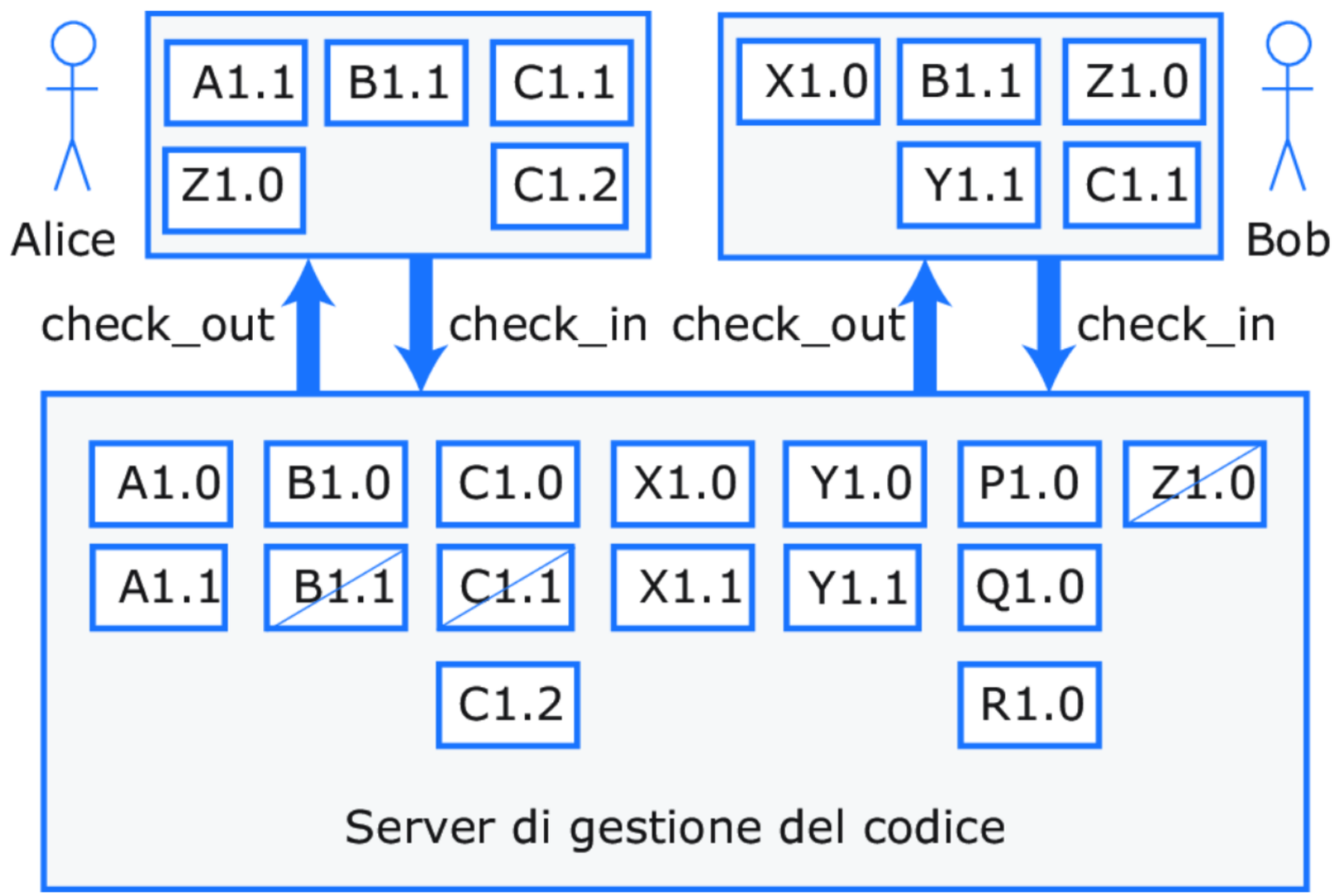
- **Trasferimento del codice** : da repository personali al sistema di gestione del codice condiviso.
- **Memorizzazione e recupero delle versioni** precedenti.
- **Branching e Merging:** creazione di *branch* per lo sviluppo parallelo e *merge* di modifiche su branch diversi.
- **Informazioni sulle diverse versioni** sono memorizzate e recuperabili

Caratteristiche di un sistema per code repository e code managment

- **Identificazione** (univoca) della versione e del rilascio
- Registrazione della **cronologia** delle modifiche
- **Sviluppo** indipendente
- **Check-out** simultaneo di tutti i file associati a un progetto
- Gestione **intelligente** della **memorizzazione**

Primi sistemi

Gestione centralizzata: problemi di conflitto

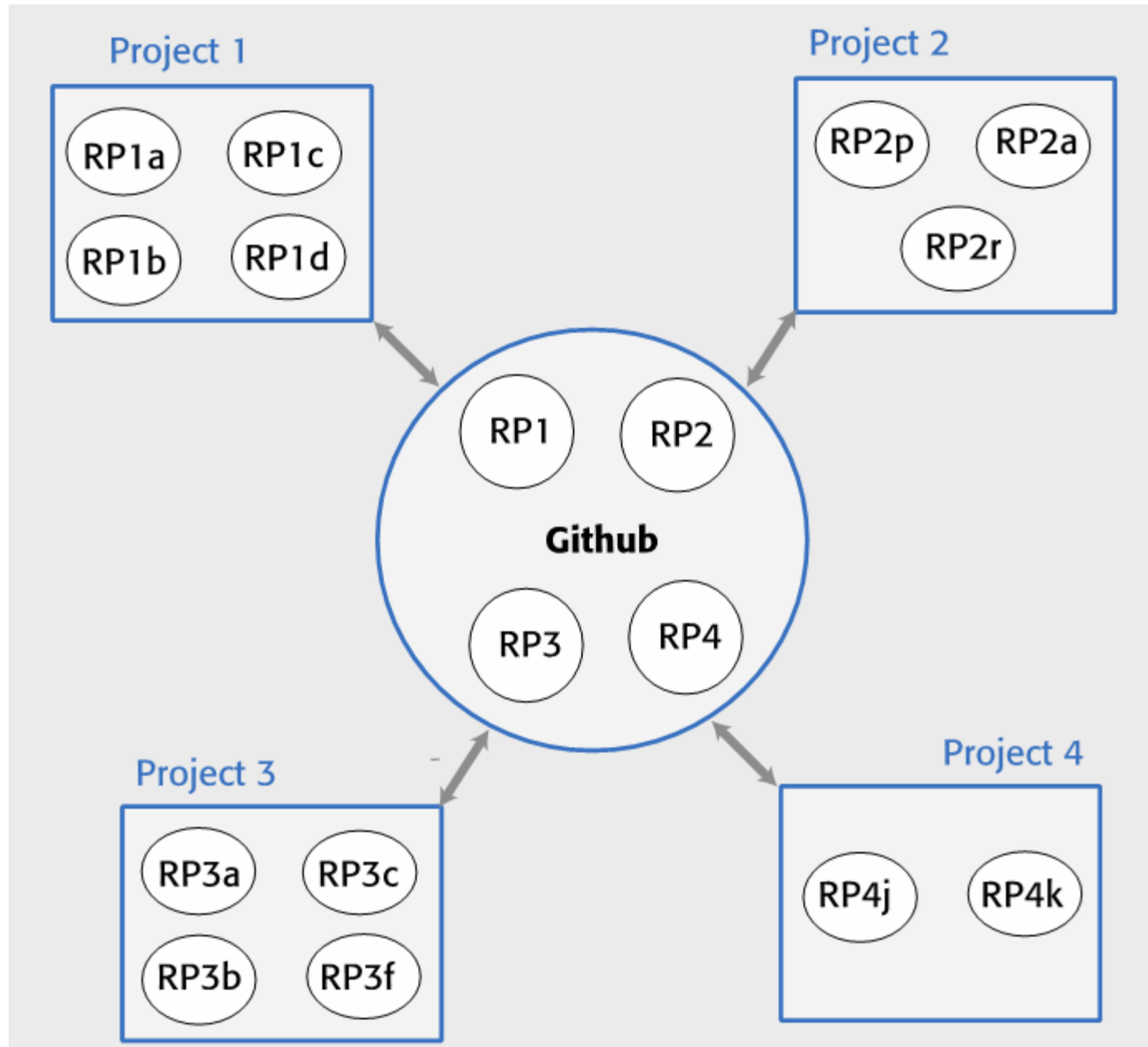


Git : gestione decentralizzata

- Sviluppato nel 2005 da Linus Torvalds per la gestione del codice sorgente del kernel Linux.
- Inizialmente orientato a supportare lo sviluppo open source su larga scala.
  - Grazie a ridotti costi di archiviazione
- Mantiene un *clone* del repository sul computer di ogni utente
- **Git**: sw per fare il **versionamento del codice e dei file** di un progetto
- **GitHub**: **sito web dove viene ospitato il codice** gestito da Git

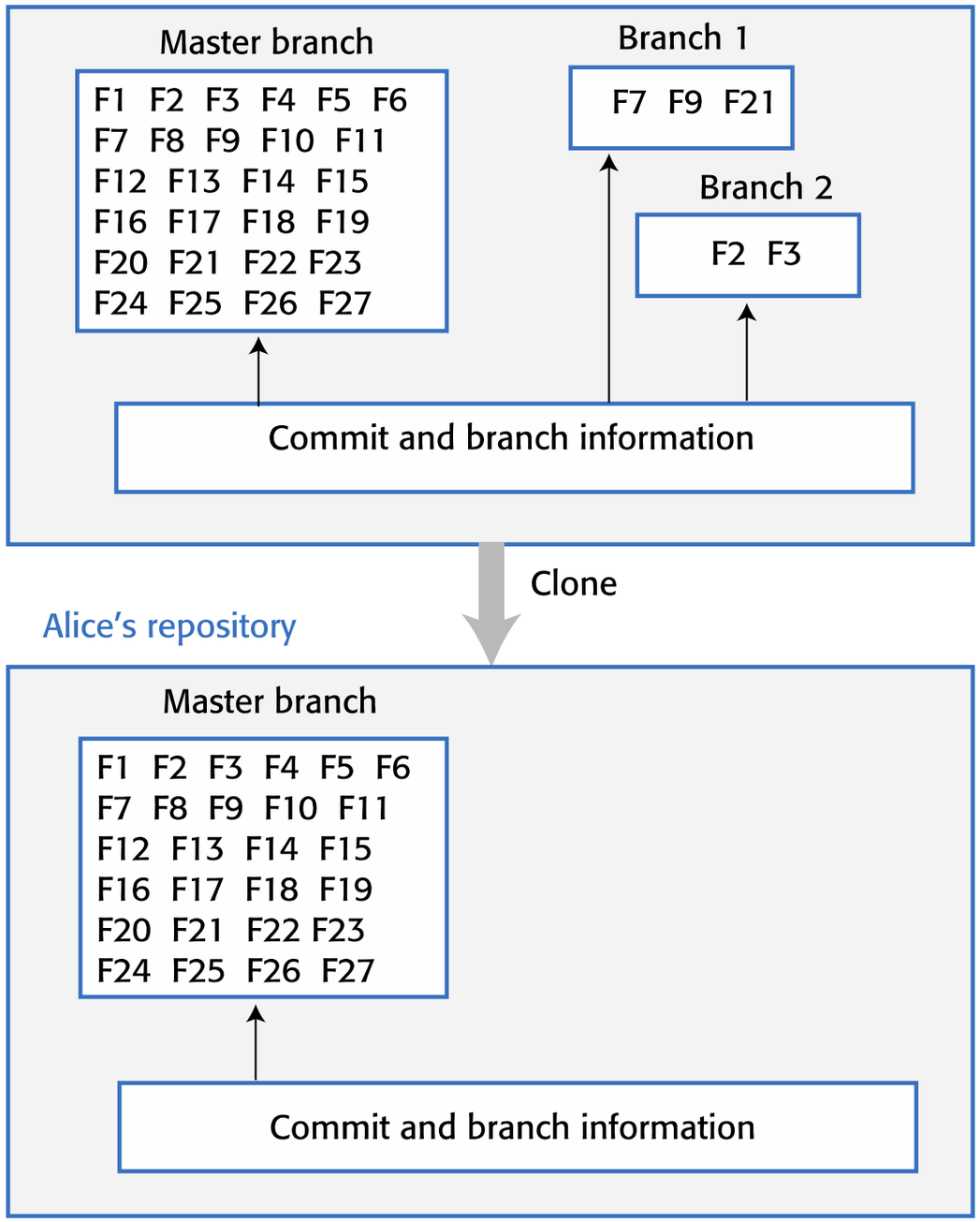
Repository Git

- Repository locali e repository esterno
- Possibilità di gestire repository di progetti condivisi da utenti diversi
  - Utente *a* lavora ai progetti P1, P2, P3 ed ha copia locale dei repository relativi



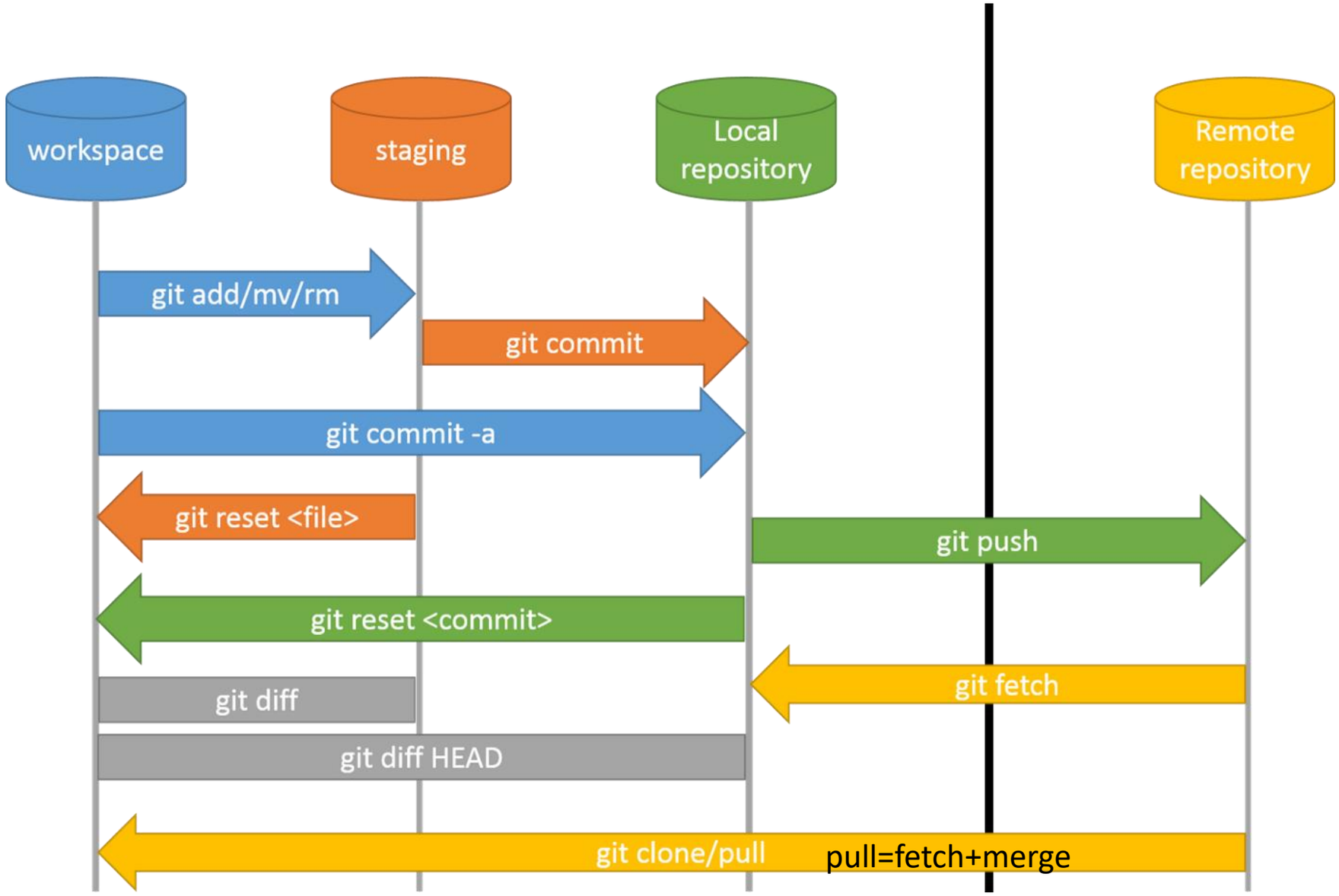
Clonazione del repository in Git

- L'utente Alice accede a repository locale che è *clone* del repository esterno
- È possible anche scaricare branch esterni
- Clone copia i file master del repo nel workspace e crea repo locale



20

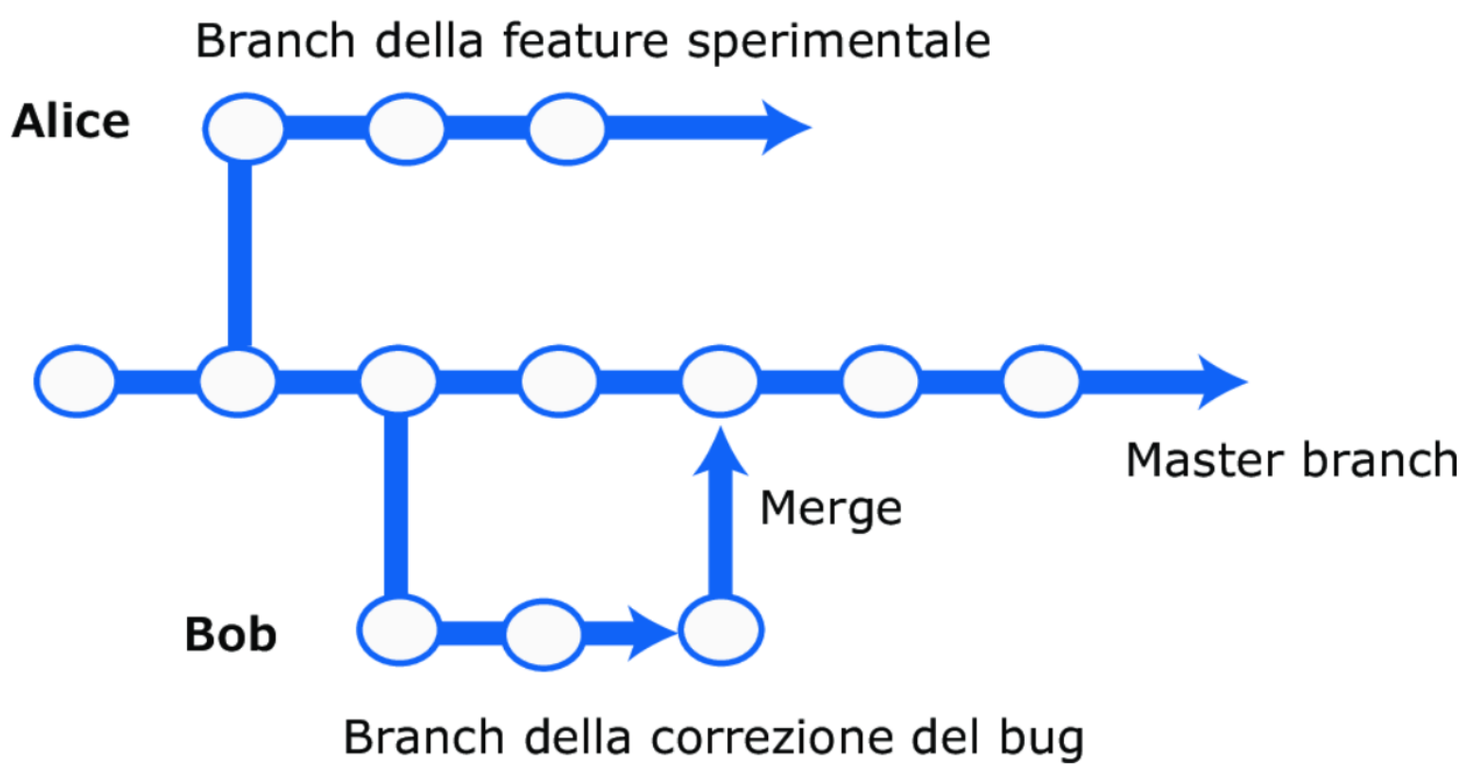
Uso di Git


workspace
staging
Local repository
Remote repository
git add/mv/rm
git commit
git commit -a
git reset <file>
git push
git reset <commit>
git fetch
git diff
git diff HEAD
git clone/pull
pull=fetch+merge

Branching e merge

Due idee fondamentali che Git supporta come tutti i sistemi di gestione del codice in DevOps

- **Branch**: versione indipendente e autonoma che viene creata quando uno sviluppatore desidera modificare un file

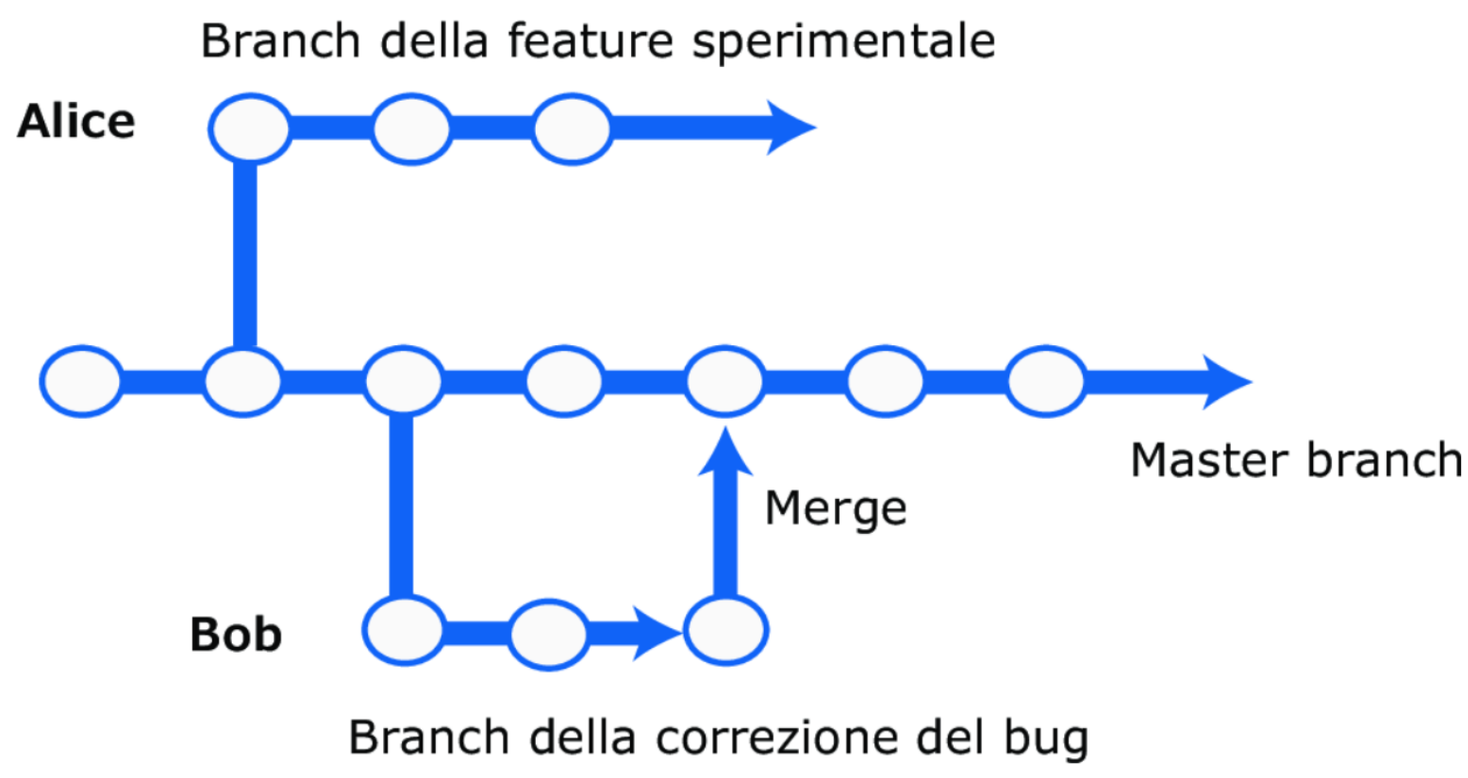


Branching e merge

Le modifiche sui branch individuali possono essere unite al master branch:

- **Merge** del branch individuale al master a seguito di *pull request*
  - *gestita dall'utente o da altro responsabile*
- **Merge** del master branch al branch individuale per allineare le versioni

Add+commit + push
= merge intrinseco
in assenza di branch

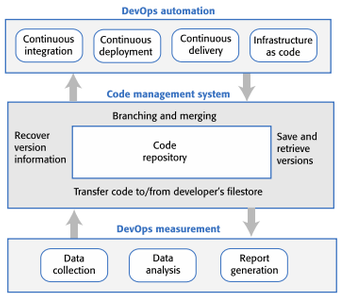


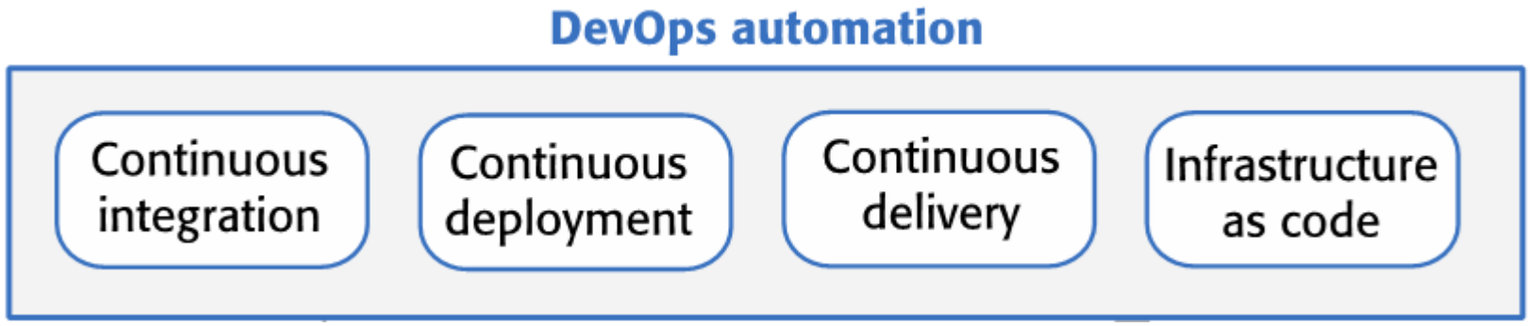
24

Vantaggi della gestione del codice distribuito tramite Git

- **Resilienza**
  - copia locale del repository (repository locale e repository esterno)
  - Ripristino del repository condiviso dai cloni locali in caso di danno o attacco informatico
  - Lavoro offline sul repository locale in mancanza di una connessione di rete
- **Velocità**
  - commit veloce delle modifiche al repository locale senza richiedere il trasferimento dei dati sulla rete
- **Flessibilità**
  - sperimentazione locale più semplice

Aspetti dell'automazione DevOps

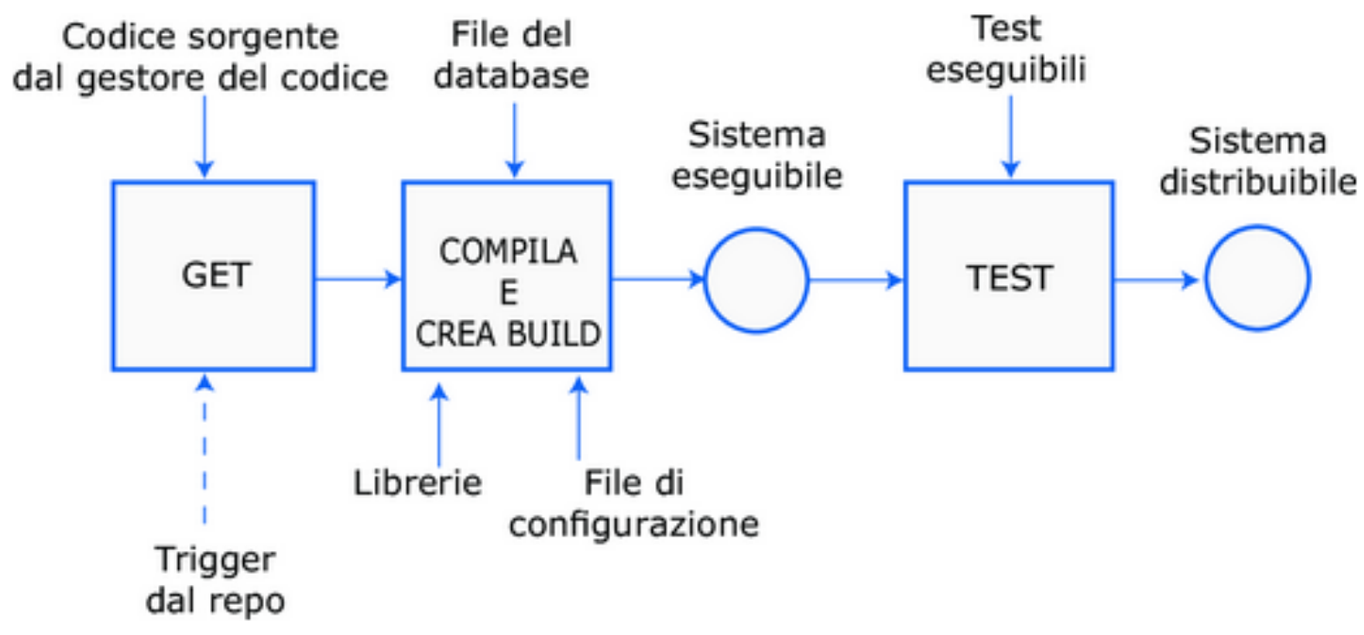




- ***Integrazione continua (CI continuos integration)***
  Ogni volta che uno sviluppatore apporta una modifica al master branch del progetto, viene creata e testata una versione eseguibile del sistema
- ***Consegna continua (Continuos delivery)***
  Viene creata una simulazione dell'ambiente operativo del prodotto e viene testata la versione del software eseguibile
- ***Distribuzione continua (Continuous deployment)***
  Una nuova versione del sistema viene messa a disposizione degli utenti ogni volta che viene apportata una modifica al master branch del software
- ***Infrastruttura come codice (Infrastructure as code)***
  I modelli dell'infrastruttura (rete, server, router, ecc.) su cui viene eseguito il prodotto vengono utilizzati dagli strumenti di gestione della configurazione per costruire la piattaforma di esecuzione del software

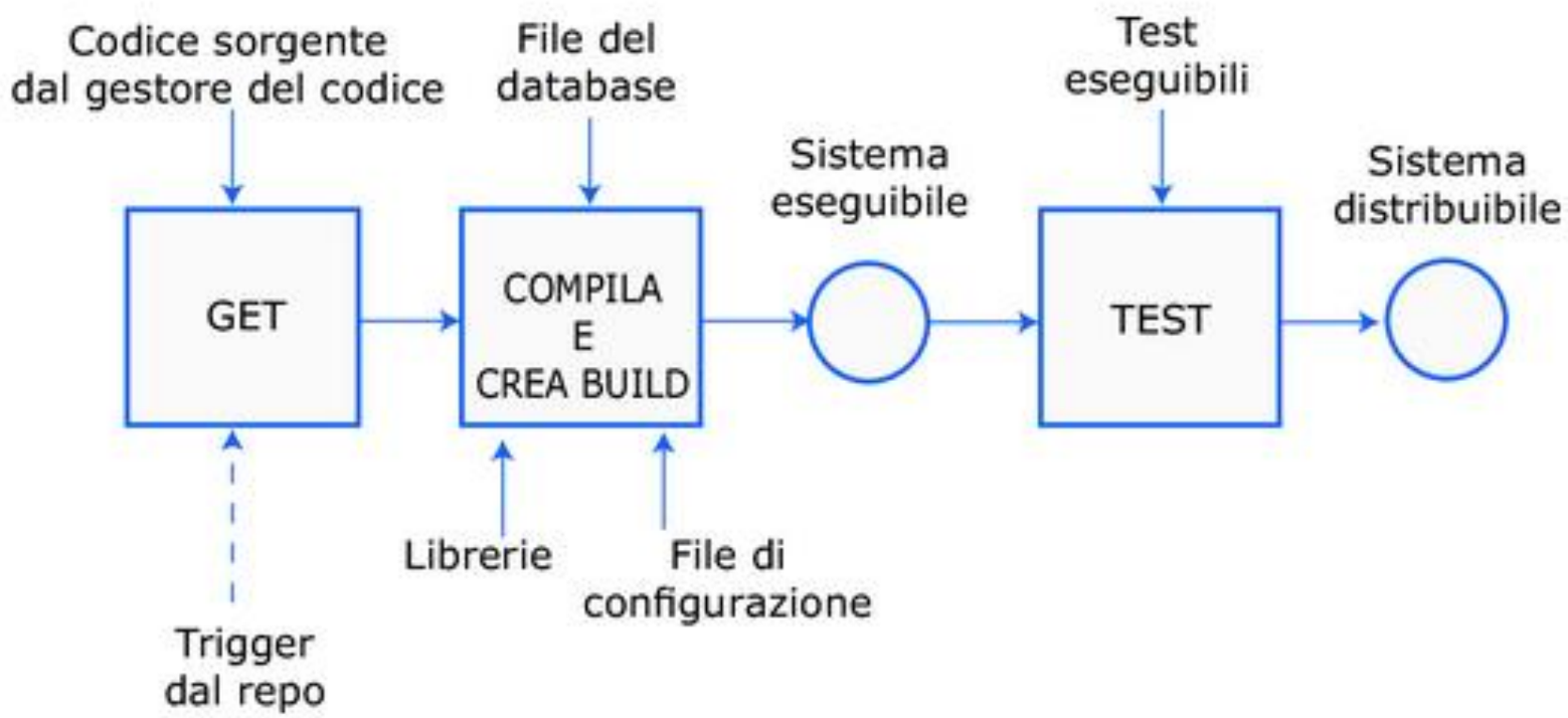
Integrazione continua

- **Integrazione continua**: una versione integrata del sistema viene creata e testata ogni volta che una modifica viene inviata al repository condiviso del sistema.
- **Pipeline di CI:**



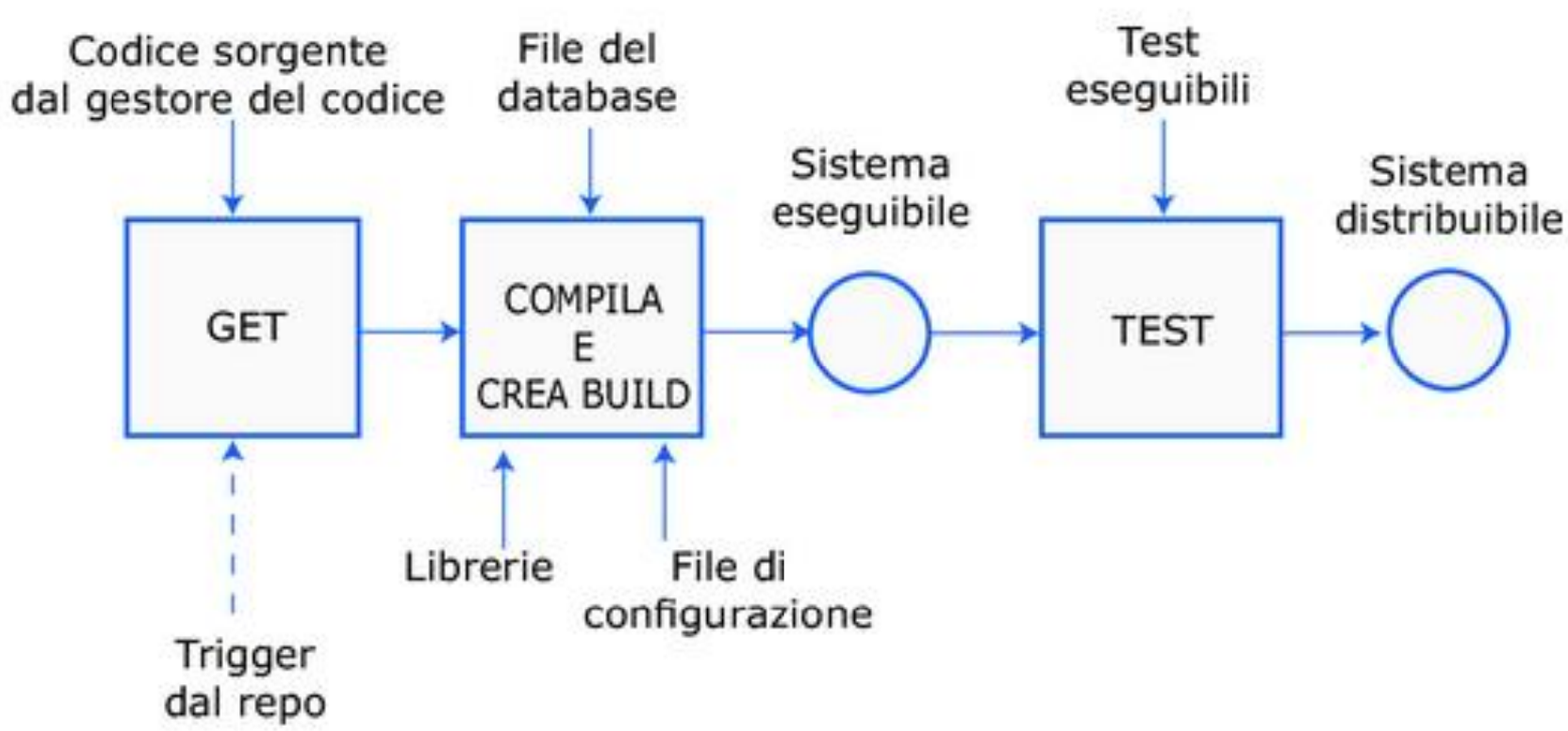
Integrazione continua

- Al termine dell'operazione *push*, il repository invia un messaggio a un server di integrazione per creare una nuova versione del prodotto



Integrazione continua

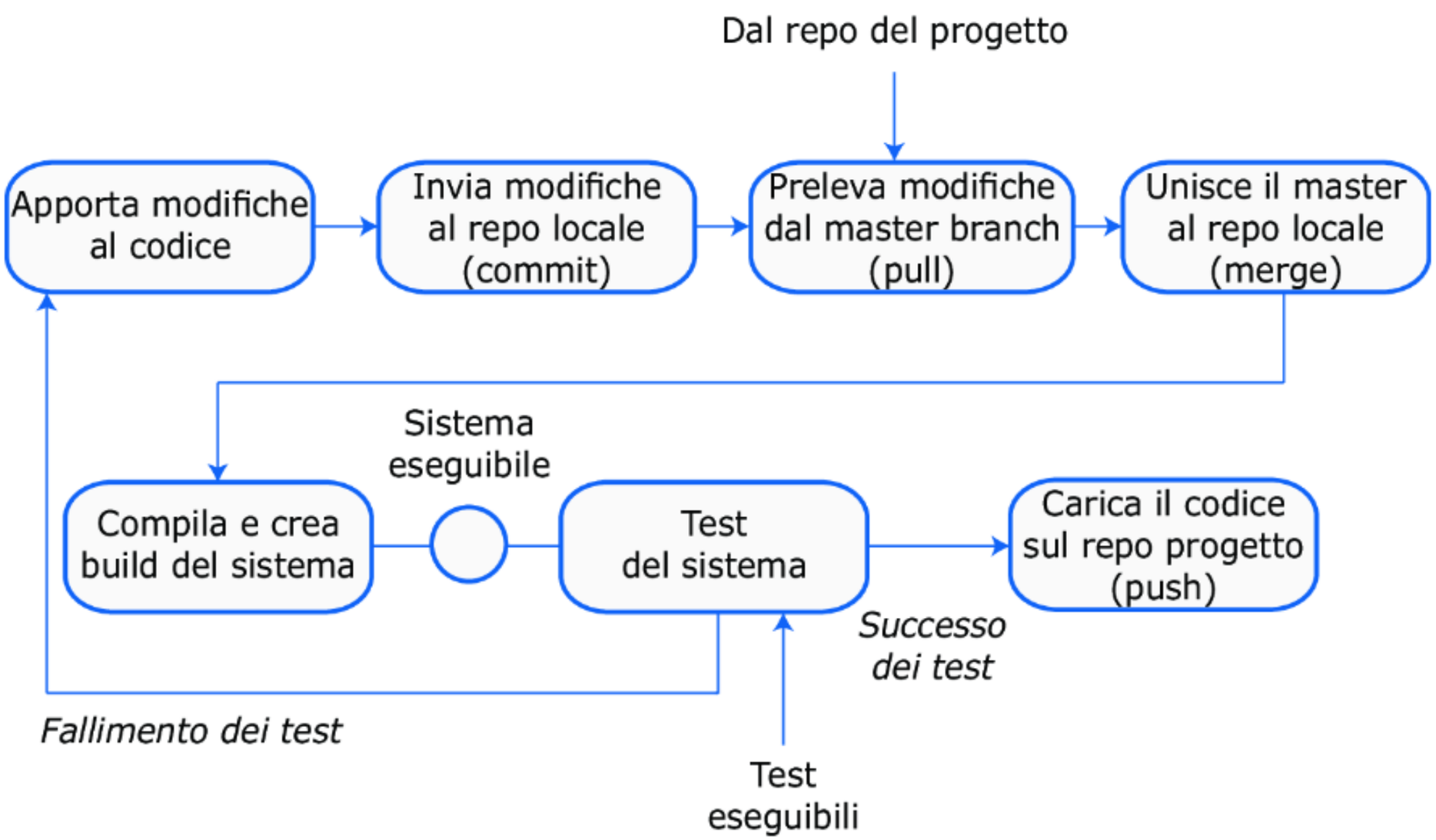
- Al termine dell'operazione *push*, il repository invia un messaggio a un server di integrazione per creare una nuova versione del prodotto (compile e build)
  - Maven per Java code



No rottura del build

- In un ambiente di integrazione continua, gli sviluppatori devono assicurarsi di non "interrompere la build".
- Rompere la build significa inserire del codice nel repository del progetto che, una volta integrato, fa fallire alcuni dei test di sistema.
- Per evitare di interrompere la build, si adotta l' approccio di "integrazione doppia" :
  - integrare e testare sul proprio computer prima di inviare il codice al repository del progetto per attivare il server di integrazione

Integrazione locale



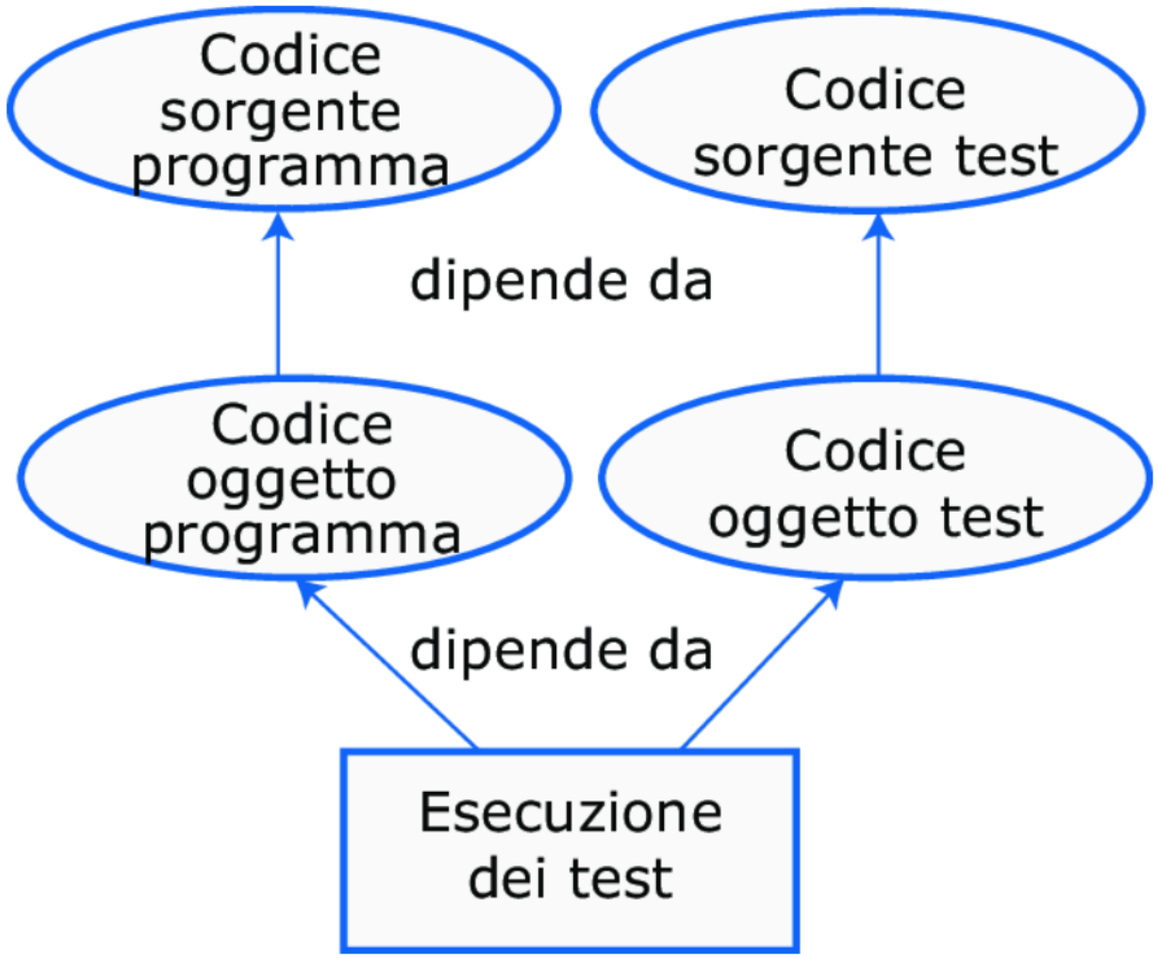
Build del sistema

- L' integrazione continua è efficace solo se il processo di integrazione è rapido e gli sviluppatori non devono *attendere i risultati dei test* del sistema integrato.
- Tuttavia, alcune attività nel processo di compilazione, come il popolamento di un database o la compilazione di centinaia di file di sistema, sono *intrinsecamente lente*.
- È quindi essenziale disporre di un *processo di compilazione automatizzato* che riduca al minimo il tempo dedicato a queste attività.

Un modello delle dipendenze

per Esecuzione di un test

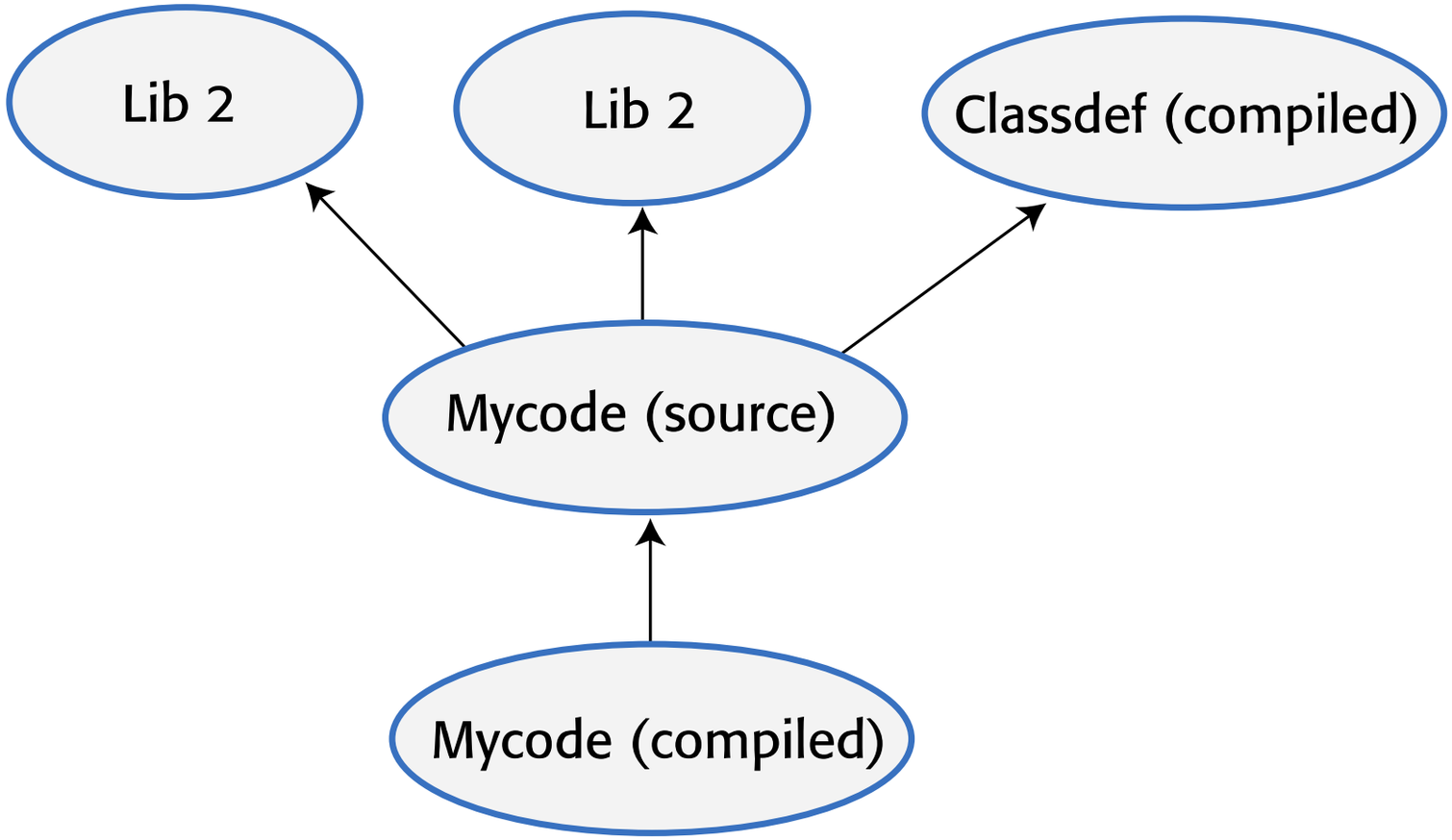
La costruzione rapida del sistema si ottiene utilizzando un processo di costruzione incrementale, in cui vengono ricostruite solo le parti del sistema che sono state modificate



Apache Ant e **Maven** hanno approcci per specificare le dipendenze

Dipendenze dei file per build rapido

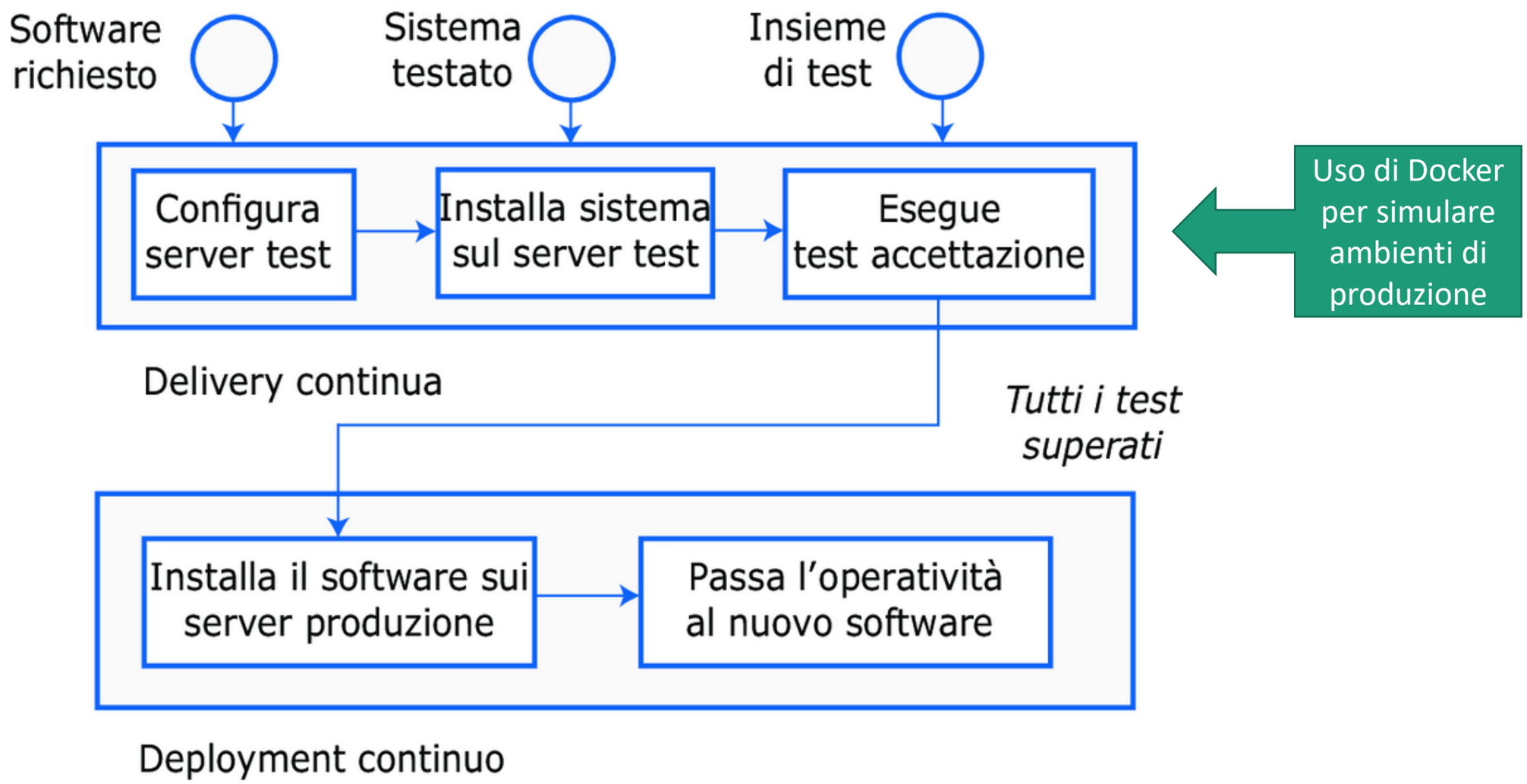
- Un sistema di build automatico utilizza la specifica delle dipendenze per capire cosa deve essere fatto.
- Utilizza il timestamp di modifica del file per decidere se un file di codice sorgente è stato modificato.



Deployment e Delivery e continui

- **Ambiente di sviluppo:** Integrazione continua fa il *build* del sistema ed *esegue i test* (delle *feature*) sul computer di sviluppo o sul server di integrazione del progetto
- Tuttavia, l'ambiente reale in cui viene eseguito il software sarà inevitabilmente diverso dal sistema di sviluppo.
- **Ambiente di produzione:** Ambiente in cui occorre testare il sw per assicurare che i fattori ambientali non causino errori di sistema o ne rallentino le prestazioni.
  - Test di carico
  - Test di throughput

Consegna e distribuzione continue

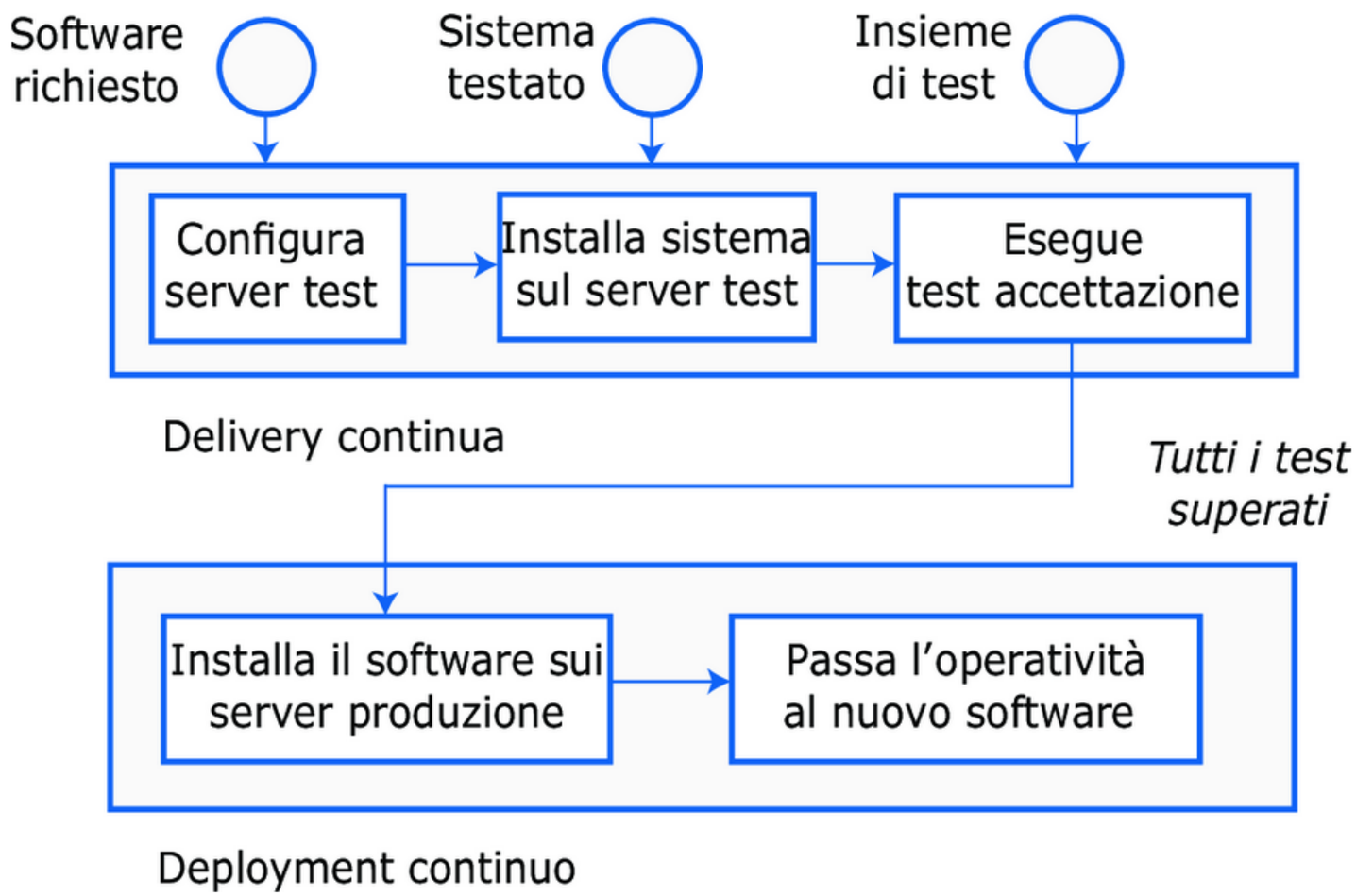


Consegna continua significa che, dopo aver apportato modifiche a un sistema, ci si assicura che il sistema modificato sia pronto per la consegna ai clienti.

Container Docker

- Usati come ambienti di produzione per esecuzione di sw prima del deployment
- Un Docker fornisce un ambiente di esecuzione autonomo in esecuzione su un sistema operativo
- Elaborando un Dockerfile si crea un'immagine del contenitore eseguibile
- Un Dockerfile permette di specificare:
  - l'infrastruttura software come codice
  - Il processo di installazione di software nel Docker

Consegna e distribuzione continue



Deployment continuo è deployment del sistema come servizio su cloud.

- Richiede interruzione momentanea delle richieste di servizio, elaborazione delle transazioni in sospeso. Poi si passa alla nuova versione del sistema

Vantaggi della distribuzione continua

- **Costi ridotti**
  grazie a pipeline di distribuzione completamente automatizzata
- **Risoluzione più rapida dei problemi**
  Se si verifica un problema, probabilmente riguarderà solo le parti aggiornate del sistema
- **Feedback dei clienti più rapido**
  da usare per modifiche correttive e perfettive
- **Testing A/B**
  opzione possibile se si dispone di un'ampia base di clienti e si utilizzano diversi server per l'implementazione.
  - È possibile distribuire una nuova versione del software su alcuni server e lasciare la versione precedente in esecuzione su altri.
  - Si misura e valuta come vengono utilizzate le nuove funzionalità per vedere se rispondono alle aspettative

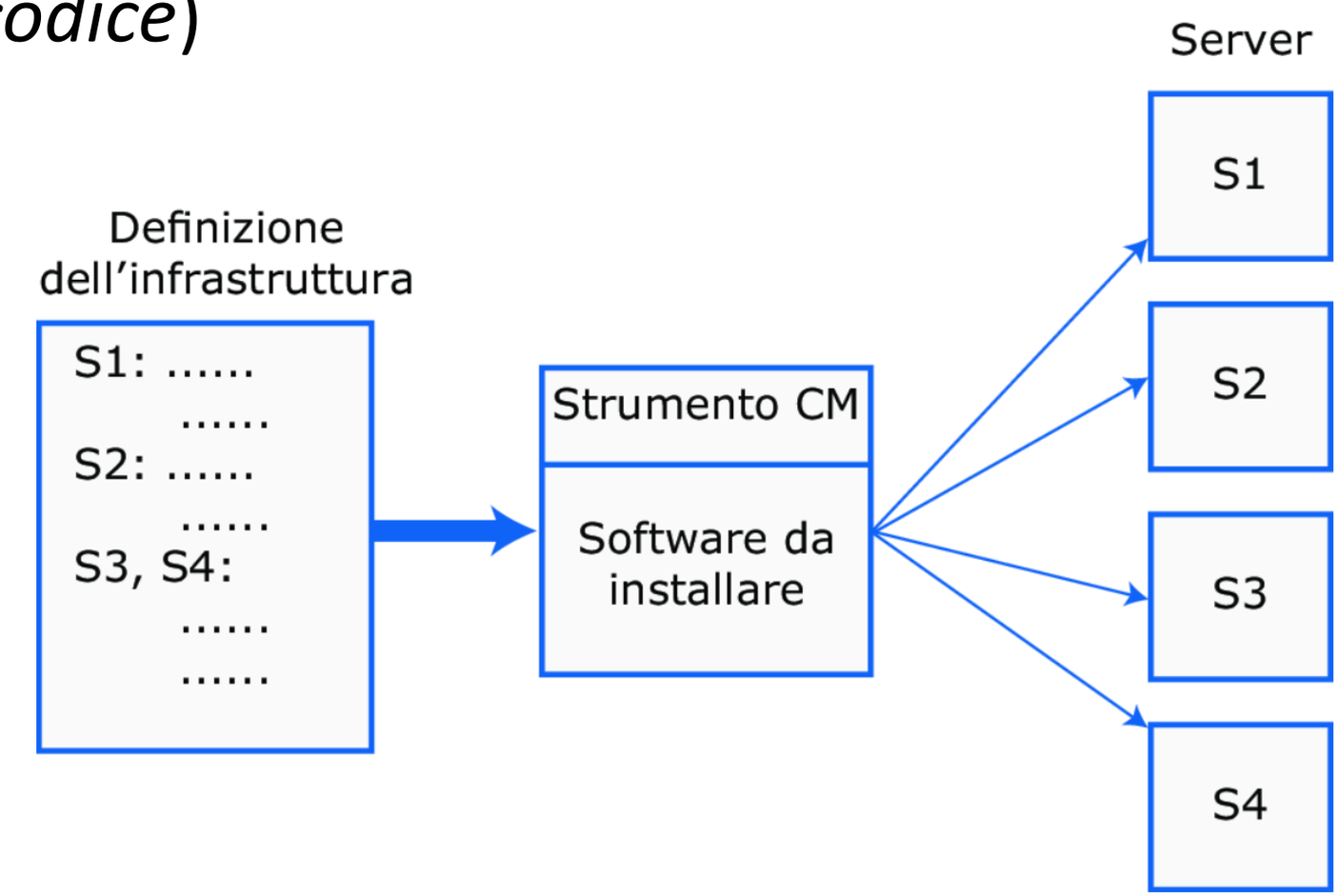
Infrastruttura come codice

- L'infrastruttura SW di un'azienda può essere complessa
- Un'azienda dispone di diversi server fisici o virtuali
  - server web, server di database, file server, ecc
  - Ogni server eroga servizi diversi
  - Ogni server ha una sua propria configurazione ed esegue pacchetti sw propri
- È difficile tenere traccia del software installato su ciascuna macchina ed aggiornare sw/configurazioni
- **Infrastruttura come codice (IaC):** proposta per risolvere il problema di *configuration managment* (CM)
  - No aggiornamento manuale del sw sui server
  - Automatizzazione del processo tramite un *modello dell'infrastruttura scritto in un linguaggio elaborabile* dalla macchina

Infrastruttura come codice

Uso di strumenti come Puppet e Chef per il CM

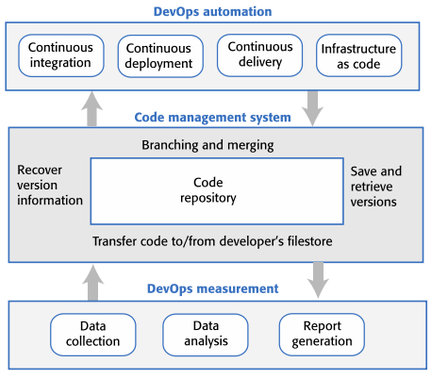
- Permettono di installare automaticamente software sui server mediante script che definiscono l'infrastruttura (*infrastruttura come codice*)



Caratteristiche e vantaggi dell'infrastruttura come codice

- **Visibilità**
  l' infrastruttura è definita come un modello autonomo che può essere compreso e rivisto dall'intero team DevOps.
- **Riproducibilità**
  L'utilizzo di uno strumento di gestione della configurazione significa che le attività di installazione verranno sempre eseguite nella stessa sequenza in modo da creare sempre lo stesso ambiente.
- **Affidabilità**
  L'automazione del processo evita errori connessi alla gestione manuale di amministratori di sistema.
- **Ripristino**
  Come qualsiasi altro codice, il modello dell'infrastruttura può essere versionato e memorizzato in un sistema di gestione del codice. Se le modifiche all'infrastruttura causano problemi, è possibile ripristinare una versione precedente e reinstallare l'ambiente precedente.

Misurazione di DevOps



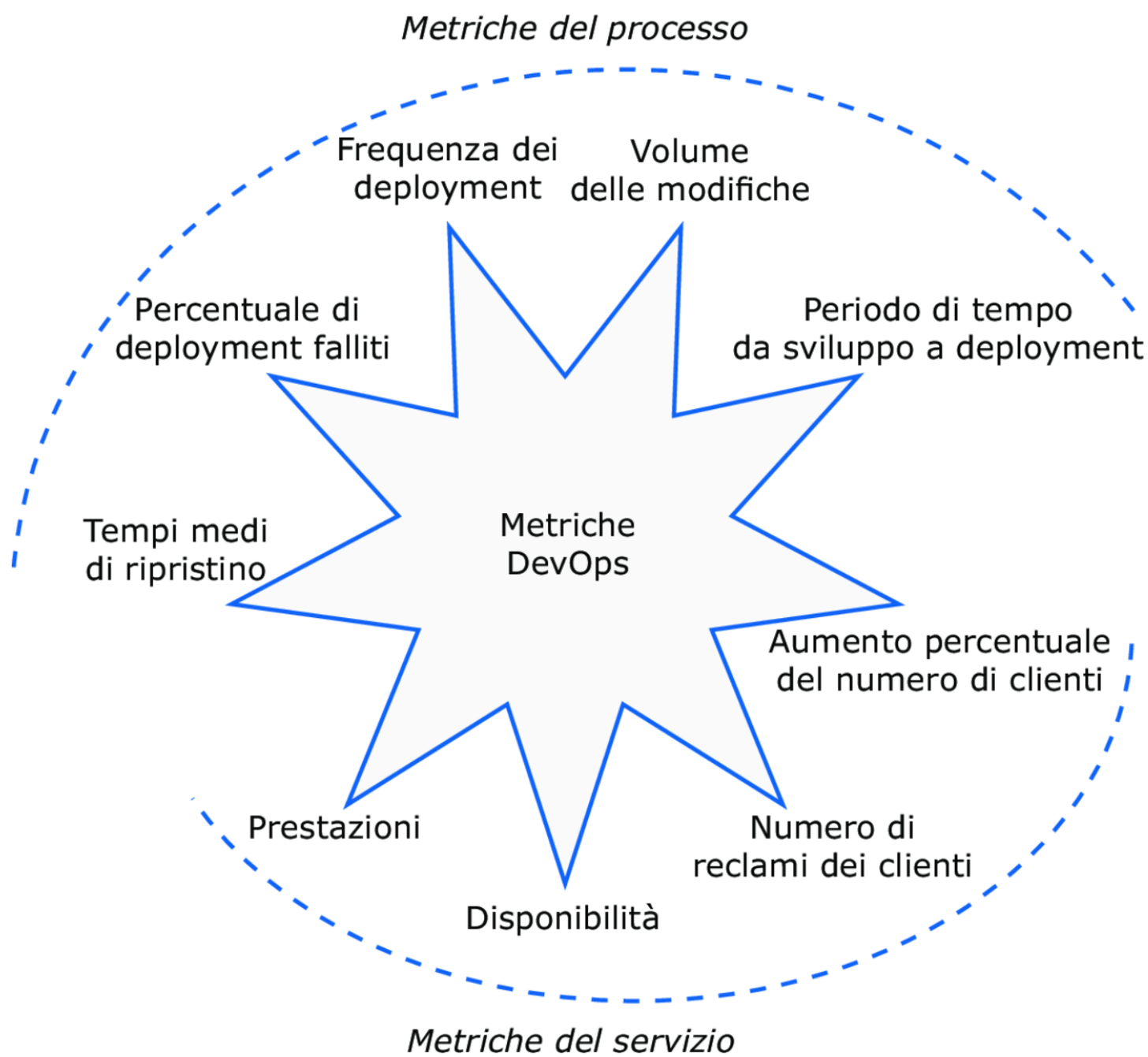
- DevOps è soggetto a continuo migloramento per una distribuzione più rapida di software di migliore qualità
- Esistono 4 tipi di misurazione dello sviluppo software:
  - **Misurazione del processo** Raccolta e analisi dei dati sul processo di sviluppo, test e implementazione.
  - **Misurazione del servizio** Raccolta e analisi dei dati sulle prestazioni, l'affidabilità e l'accettabilità del software per i clienti.
  - **Misurazione dell'utilizzo** Raccolta e analisi dei dati su come i clienti utilizzano il prodotto.
  - **Misurazione del successo aziendale** Raccolta e analisi dei dati su come il prodotto contribuisce al successo complessivo dell'azienda.

Misurazione automatizzata

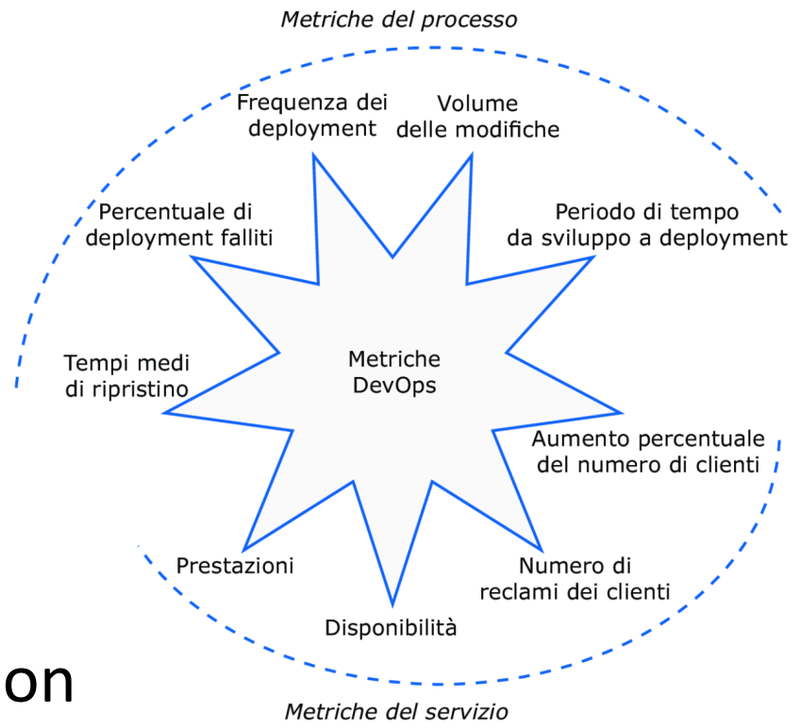
- Il principio di DevOps di automatizzare tutto dovrebbe essere applicato anche alla misurazione del software.
- DevOps deve disporre di meccanismi per
  - istrumentare il software per raccogliere dati su se stesso
  - monitoraggio per raccogliere dati sulle prestazioni e la disponibilità del software.
- La misurazione del processo non è semplice

Metriche utilizzate nella scorecard DevOps

Payal Chakravarty di IBM suggerisce un approccio pratico alla misurazione DevOps basato su una scorecard per le 9 metriche



Scheda delle metriche

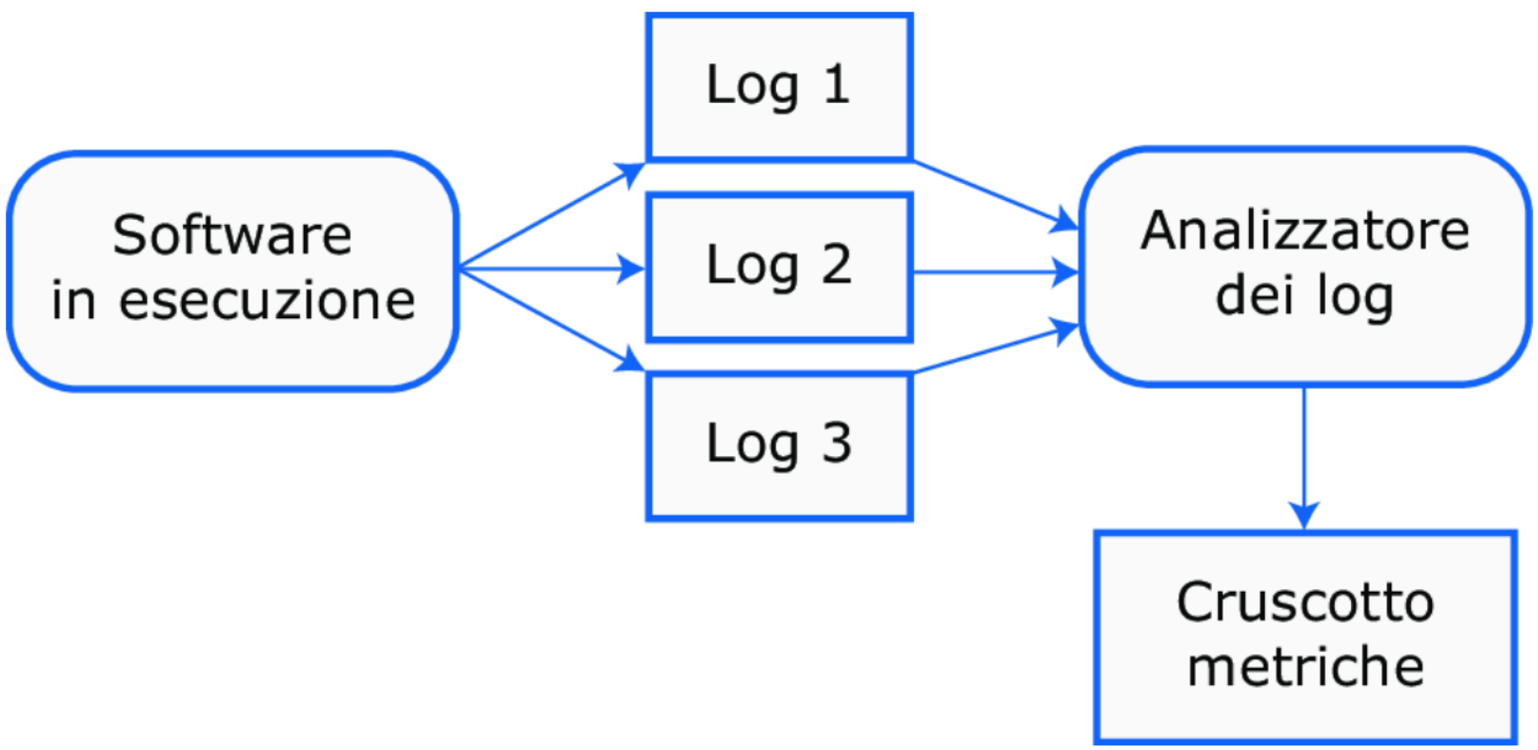


Includono metriche di processo e metriche di servizio

- Obbiettivo per le metriche di processo:
  - Diminuzione del numero di distribuzioni non riuscite, del tempo medio di ripristino dopo un errore del servizio, del tempo tra sviluppo e distribuzione.
  - Aumento nella frequenza di distribuzione e nel numero di righe di codice (nuovo o modificato) che vengono spedite.
- Obbiettivo per le metriche del servizio:
  - Diminuzione del numero di reclami dei clienti
  - Aumento della disponibilità, delle prestazioni, del numero di nuovi clienti

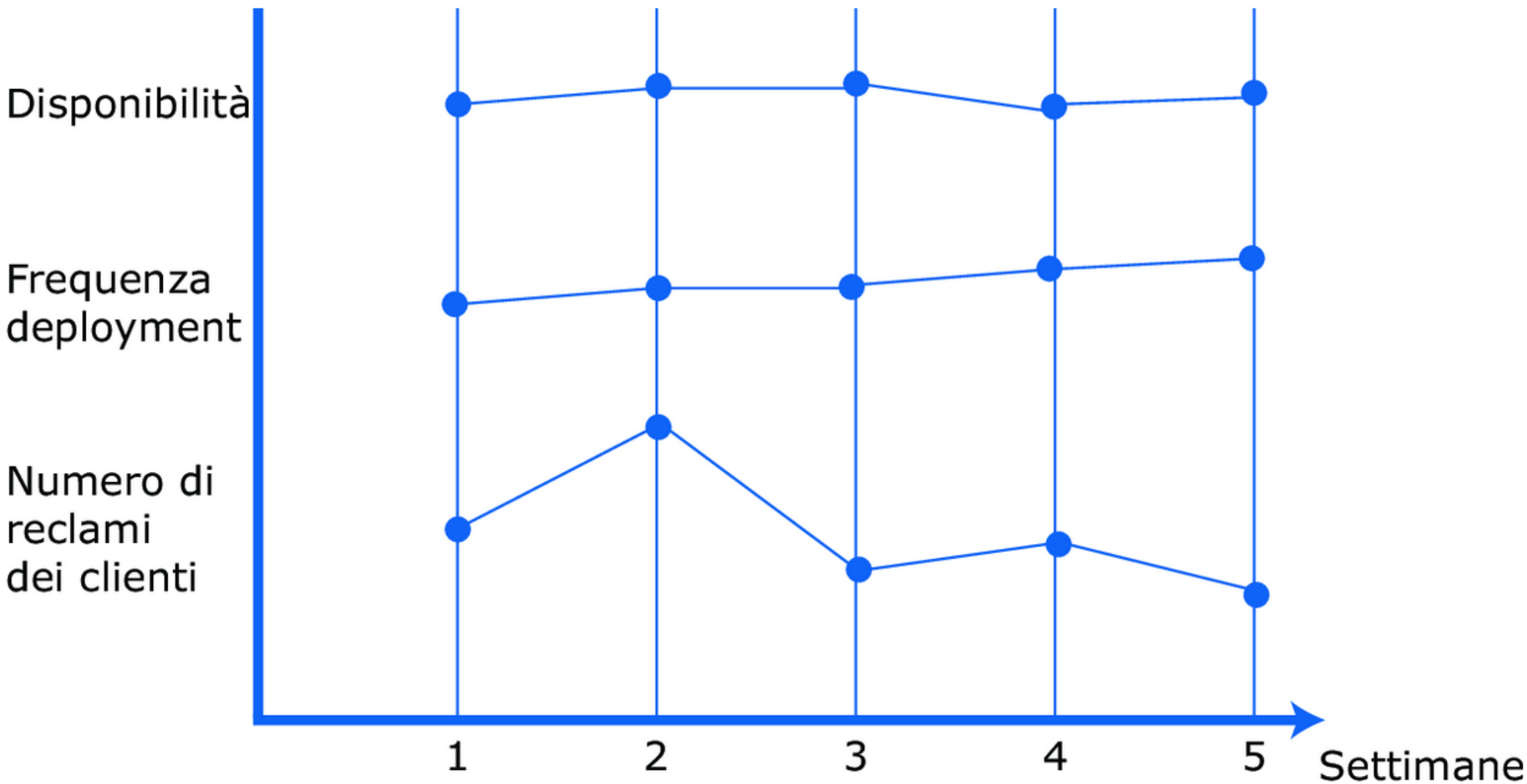
Registrazione e analisi

Si utilizzano strumenti di raccolta dei dati

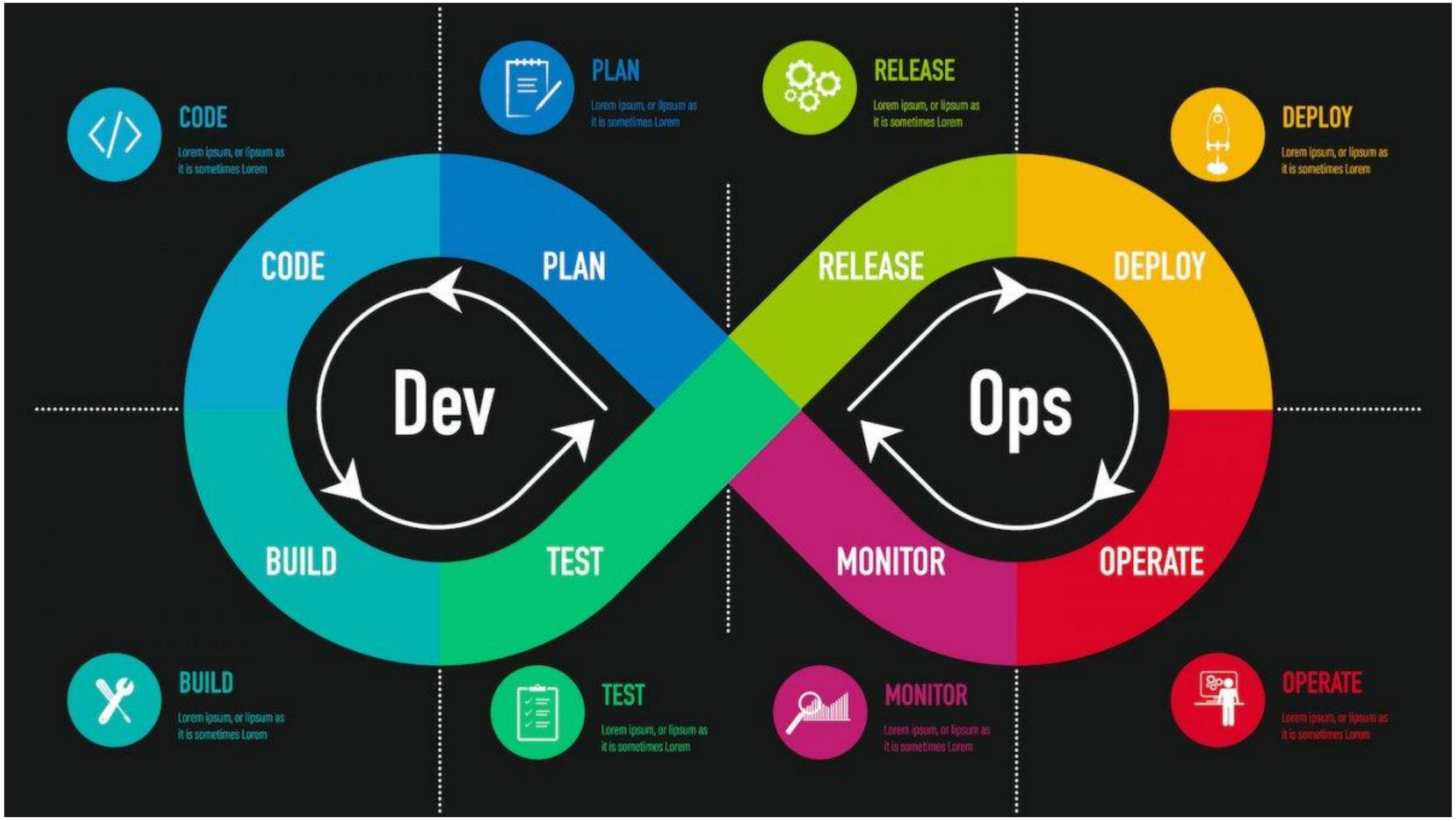


Trend delle metriche

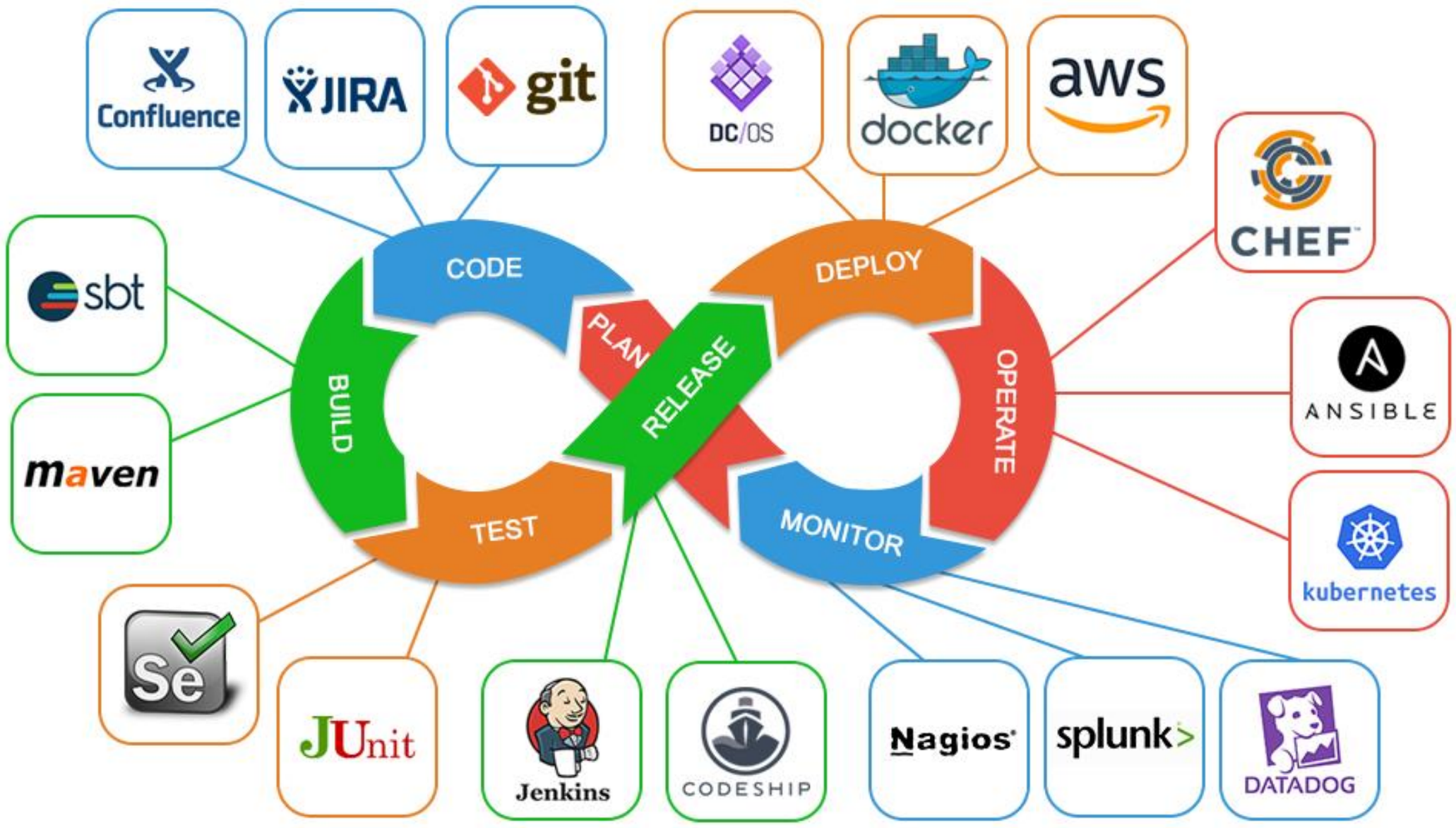
Si utilizzano strumenti per il report periodico dei dati



L’8 di DevOPs


CODE
Lorem ipsum, or lipsum as it is sometimes Lorem
PLAN
Lorem ipsum, or lipsum as it is sometimes Lorem
RELEASE
Lorem ipsum, or lipsum as it is sometimes Lorem
DEPLOY
Lorem ipsum, or lipsum as it is sometimes Lorem
CODE
PLAN
RELEASE
DEPLOY
Dev
Ops
BUILD
TEST
MONITOR
OPERATE
BUILD
Lorem ipsum, or lipsum as it is sometimes Lorem
TEST
Lorem ipsum, or lipsum as it is sometimes Lorem
MONITOR
Lorem ipsum, or lipsum as it is sometimes Lorem
OPERATE
Lorem ipsum, or lipsum as it is sometimes Lorem

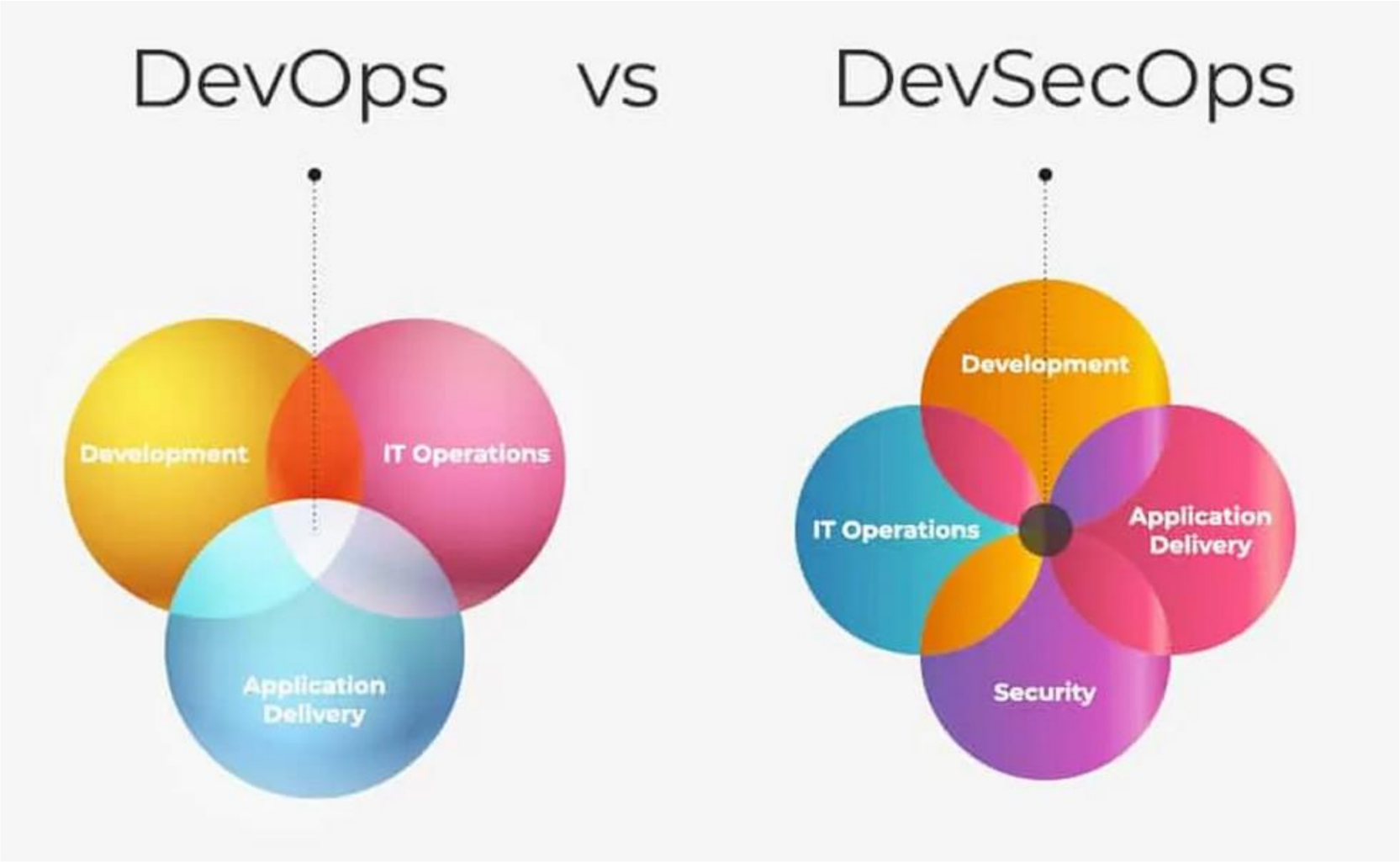
Strumenti e strategie di DevOps



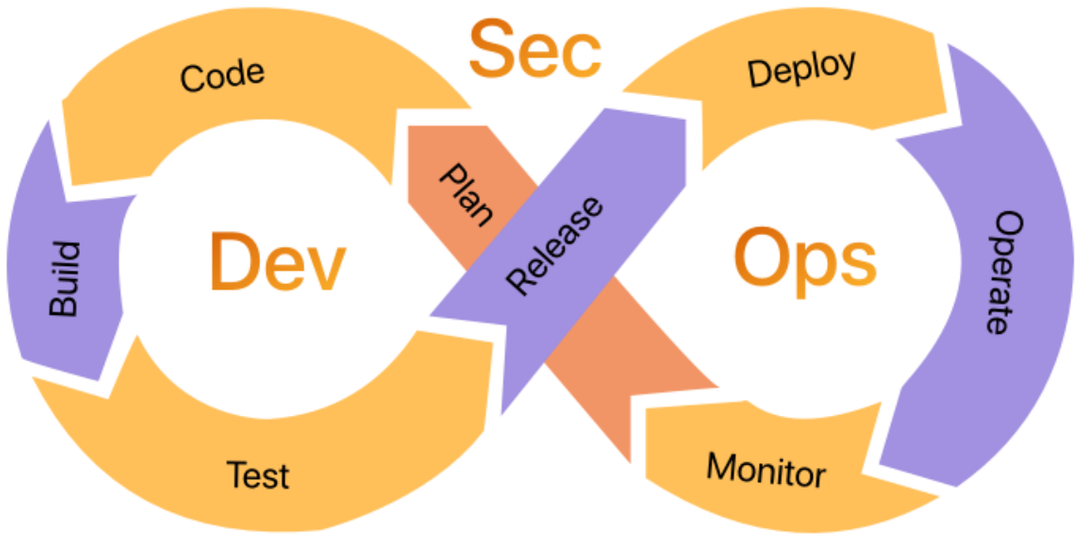
DevOps e DevSecOps/SecDevOps

SecDevOps = DevOps + **security by-design**

- definizione di policy +penetration test

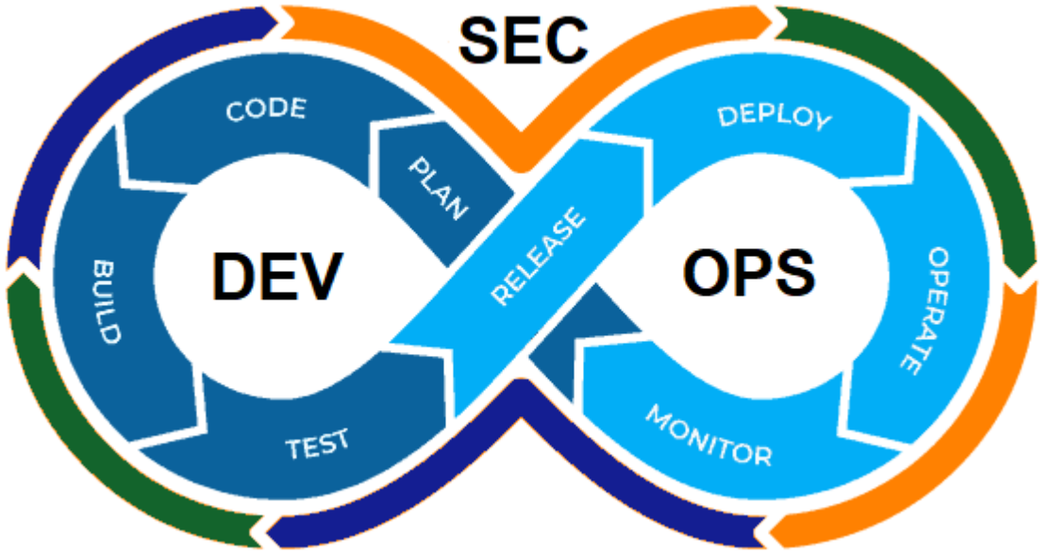


DevSecOps vs SecDevOps



Peter: *Hey, we should call it DevSecOps. Security should be in the middle of DevOps, we say "you build it, you run it". Now, it should be: "you build it, you secure it".*

Paul: *I think it should be better to call it SecDevOps. Security should be always first. And, as they say, we have to "push left security".*



Mary: *Why don't we just leave it as DevOps? Security should be intrinsic to both development and operations, and we don't need to create a new term to strengthen the idea that Security should be everywhere… "Security by design", "Security is everyone's responsibility"*